

CS4269

Wang Xiyu A0282500R

Contents

1	Syntax of Propositional Logics	3
1.1	Well-formed Formula (WFF)	3
1.2	Closed Set	3
1.2.1	All Closed Set	3
1.3	FORM is the minimum of the set of Closed	4
1.4	Subformula	4
2	Semantics of Propositional Logics	5
2.1	Truth Assignment/Proposition evaluation	5
2.2	Entail/Model	5
2.2.1	Valuation and Model	5
2.2.2	$\models$ as a relation	5
2.2.3	Logical Equivalence	6
2.3	Satisfiability of Formula	6
2.3.1	Valid formula(tautology)	6
2.4	Satisfiability of a set of formula	7
2.4.1	Relevance	7
2.4.2	Logical consequence	7
2.5	Modelling Computations as Propositional Logic	8
2.6	Encoding a Full Adder in Pure Propositional Logic	11
3	Compactness Theorem for Propositional Logics	13
3.1	Application of the Compactness theorem for proposition logics	16
3.2	Proof of König's Lemma via Compactness	18
4	Proof System	18
4.1	Hilbert Style Proof System	18
4.2	Metatheorems	21
4.2.1	Monotonicity theorem	21
4.2.2	Deduction	21
4.3	Weak Completeness and Completeness of HSPS	23
4.4	Resolution Proof System	24
5	Complexity	27
5.1	Turing Machine	27
5.2	Semantics of TM	28
5.3	Running time and Space	30
5.4	Complexity classes	30
5.5	Hardness	31
5.6	Reduction	31
5.7	Cook-Levin Theorem	32
6	Syntax of First Order Logics	34
6.1	Vocabulary of FOL	34
6.2	Modelling with FOL	35
7	Semantics of First Order Logics	35

8	Compactness of First Order Logics	39
8.1	Prenex Normal Form	39
8.2	Skolemization	41
8.3	Proof of the compactness theorem for FOL	44
8.3.1	Γ is satisfiable $\implies Gr(\Gamma)$ is propositionally satisfiable	44
8.4	Special FOL fragments	46
8.4.1	Quantifier-free FOL	46
8.5	Homomorphism and Isomorphism	47
8.6	Theory	47
8.6.1	Common clauses used to define theories	48
8.6.2	Monotonicity under theory inclusion	48
8.6.3	Theory of a structure	48
8.7	Nelson-Oppen Theory Combination	50
8.7.1	The Nelson-Oppen Algorithm	50
8.8	Complexity and Completeness	51
9	Computability	51
9.1	Halting problem	51
9.2	Reduction	53
9.3	RE-hardness	54
9.4	Validity of FOL	54
10	Second Order Logics	55
10.1	Syntax of Second Order Logics	55
10.2	Expressiveness of SOL	55
10.2.1	Finite structures	55
10.2.2	Reachability	56
10.2.3	Cyclic graph	57
10.2.4	Even	57
10.2.5	Bipartite graph	57
10.2.6	Connectedness	58
10.2.7	Well-Ordering	58
10.2.8	Hamiltonian Graphs	58
10.2.9	Well-Foundedness	58
10.2.10	Graph Isomorphism	58

1 Syntax of Propositional Logics

Definition 1.1. *DSyntax decides what are allowed to be written*

1.1 Well-formed Formula (WFF)

A formula ψ is obtained by composing $p \in P$ with $c \in C$, where P is the set of propositions and C is the set of connectors. here we use any set of complete logic, like $C = \{\neg, \vee\}$. This is not a precise definition as counter examples that do not form a valid formula can be formulated based on this, for example $\neg \vee p$ means nothing. To formalize the definition we may attempt to define a language with the following grammar:

$$\Sigma = P \cup C \cup \{ (,) \}$$

$$FORM : \{ \psi = p \in P \mid \neg\psi \mid \psi \vee \psi \mid (\psi) \}$$

However this is a recursive definition as the definition of the a formula ψ involves itself. We need to find a declarative way to define this recursive/inductive construction.

Definition 1.2. *FORM is the smallest set of strings over Σ where*

1. $P \subseteq FORM$
2. $\forall \psi \in FORM, \neg\psi \in FORM$
3. $\forall \psi_1, \psi_2 \in FORM, \psi_1 \vee \psi_2 \in FORM$

We can see as a construction rule, we are admitting strings into the set of formula, but we need to somewhat impose restriction on what is not acceptable.

1.2 Closed Set

Let's remove the attributive **the smallest** first and define another set that contains all the set that satisfy these rules listed above:

Definition 1.3. *Closed is a set of strings over Σ where*

1. $P \subseteq Closed$
2. $\forall \psi \in Closed, \neg\psi \in Closed$
3. $\forall \psi_1, \psi_2 \in Closed, \psi_1 \vee \psi_2 \in Closed$

1.2.1 All Closed Set

And we define a set ACS (All-closed sets). We can define FORM via the following constriction:

$$FORM = \bigcap ACS$$

Suppose $BCS = \emptyset$ and from definition

$$U_0 = \bigcap BCS = \{w \in \Sigma^* \mid \forall S \in BCS, w \in S\}$$

However, $BCS = \emptyset$, The statement $\forall S \in BCS, w \in S$ is vacuously true $\forall w \in \Sigma^*$, which means that

$$U_0 = \Sigma^* = \bigcap BCS$$

A contradiction. Therefore we have proved that ACS cannot be empty, thus $\forall P, \exists S$ such that S is a closed set.

1.3 FORM is the minimum of the set of Closed

First let's prove that FORM is closed. It seems to be trivial that the intersection of closed sets is closed but formal proof is still needed. We denote a set from the intersection construction as T_0 .

Lemma 1.4. T_0 is closed

We are proving that the set T_0 we get from the intersection construction still satisfy the rules that define what is closed.

Proof. From rule 1,

$$\begin{aligned} \forall S \in ACS, P \subseteq S \\ \rightarrow \forall p \in P, \forall S \in ACS, p \in S \\ \rightarrow \forall p \in P, p \in \bigcap ACS \\ \rightarrow P \in T_0 \end{aligned}$$

From rule 2,

$$\begin{aligned} \forall S \in ACS, T_0 \subseteq S \\ \text{let } w \in T_0, \forall S \in ACS, w \in S, \neg w \in S \\ \rightarrow \forall w \in T_0, \neg w \in \bigcap ACS \\ \rightarrow \forall w \in T_0, \neg w \in T_0 \end{aligned}$$

From rule 3,

$$\begin{aligned} \forall S \in ACS, T_0 \subseteq S \\ \text{let } w_1, w_2 \in T_0, \forall S \in ACS, w_1 \vee w_2 \in S \\ \rightarrow \forall w_1, w_2 \in T_0, w_1 \vee w_2 \in \bigcap ACS \\ \rightarrow \forall w_1, w_2 \in T_0, w_1 \vee w_2 \in T_0 \end{aligned}$$

□

Lemma 1.5. T_0 is a smallest Closed set

Proof. The proof of minimality is trivial. $T_0 = \bigcap ACS \rightarrow \forall S \in ACS, T_0 \subseteq S$

□

Lemma 1.6. T_0 is the smallest Closed set (unique). In other word,

$$\forall T'_0 \in \{T \mid \forall S \in ACS, T \subseteq S\}, T'_0 = T_0$$

Now we can use the generative rule to define formula, which is called **Backus-Naur form**. Note that a context-free grammar is a type of formal language. Backus Naur form is a specification language for this type of grammar. It is used to describe language syntax.

1.4 Subformula

Definition 1.7. ϕ_1 is a subformula of ϕ_2 , denoted by

$$\phi_1 \preceq \phi_2$$

as a relation is

- reflexive: $\forall \phi \in FORM, \phi \preceq \phi$
- transitive: $\forall \phi_1, \phi_2, \phi_3 \in FORM, (\phi_1 \preceq \phi_2) \wedge (\phi_2 \preceq \phi_3) \rightarrow \phi_1 \preceq \phi_3$

Inductive definition:

$$\begin{aligned} \forall \phi \in FORM, \phi \preceq (\neg \phi) \\ \forall \phi_1, \phi_2 \in FORM, \phi_1 \preceq (\phi_1 \vee \phi_2) \\ \phi_2 \preceq (\phi_1 \vee \phi_2) \end{aligned}$$

We might intuitively think that subformula can be defined using the notion of substring, since we have been considering formula as concatenation of proposition and connector symbols. However based on our inductive definition, we can come up with the following counter example to a definition using substring: A formula ϕ_1 is a subformula of ϕ_2 if and only if ϕ_1 is a well-formed formula, and is a substring to ϕ_2

$$p \vee q \not\subseteq \neg p \vee q$$

What if we tighten the syntax by strictly enforce the use of parentheses whenever we can

2 Semantics of Propositional Logics

Definition 2.1. *Semantics describe meanings.*

2.1 Truth Assignment/Proposition evaluation

A truth assignment or a valuation of propositions is a function that maps from the set of propositions to true and false.

$$\begin{aligned} v : P \rightarrow \{T, F\} \\ v \in [P \rightarrow \{T, F\}] \end{aligned}$$

Example 2.2. Consider $v = \{p_1 \mapsto T, p_2 \mapsto F\}$, we can derive

$$\begin{aligned} v(p_1) &= T \\ v(p_2) &= F \\ v(\neg p_2) &= T \\ v(p_1 \vee p_2) &= T \text{ etc.} \end{aligned}$$

2.2 Entail/Model

We denote entailment of an evaluation function to formula as

$$v \models \psi$$

We also inductively define the following rule

$$\begin{aligned} \forall p \in P, v \models p &\iff v(p) \\ v \models \psi &\iff v \not\models \neg \psi \\ v \models \psi_1 \vee \psi_2 &\iff (v \models \psi_1) \vee (v \models \psi_2) \end{aligned}$$

2.2.1 Valuation and Model

A valuation v is a model of a formula ϕ , if and only if $v(\phi) = T$.

- $\forall \phi$ over P , $\exists 2^{|P|}$ valuations.
- A formula can have many or one or none models

2.2.2 $\models$ as a relation

Definition 2.3. $\models \subseteq \{2^{|P|} \times FORM\}$ is a binary relation, between truth assignment to formula. We define as follows:

- $\forall p \in P, v \models p \iff v(p) = 1$

- $v \not\models p$ otherwise.

Consider a valuation v as a subset of P such that the propositions in v denoted by $v(p)$ is assigned with true.
 $v \in \mathcal{P}(P)$

ϕ	ψ	$\neg\phi$	$(\phi \wedge \psi)$	$(\phi \vee \psi)$	$(\phi \rightarrow \psi)$	$(\phi \leftrightarrow \psi)$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

Definition 2.4.

$$p(v) = v(p)$$

A proposition can be viewed as a circuit is just a wire, thus if a proposition is assigned with true by a valuation, the valuation evaluates this proposition to be true

Lemma 2.5. Let $\phi \in FORM, v \in \mathcal{P}(P), v \models \phi \iff \phi(v) = 1$

Proof. Base case: $\forall \phi = p \in P, v \models \phi \iff v(p) = 1 \iff p(v) = 1$
Inductive case:

1.

□

2.2.3 Logical Equivalence

Definition 2.6. Syntax definition of logical equivalence is as follows:

$$\phi_1 \equiv \phi_2 \iff \forall v : P \mapsto \{T, F\} (v \models \phi_1), v \models \phi_2$$

Example 2.7.

$$p \vee (\neg p) \equiv q \vee (\neg q)$$

$$\begin{aligned} \forall v \models (p \vee (\neg p)), \\ v \models p \text{ or } v \models (\neg p) \\ v \models p \text{ or } v \not\models p \\ \forall v : P \mapsto \{T, F\}, \end{aligned}$$

TODO

2.3 Satisfiability of Formula

Definition 2.8. Let $\phi \in FORM, \phi$ is satisfiable

$$\iff \exists v : P \mapsto \{T, F\}, v \models \phi$$

2.3.1 Valid formula (tautology)

Definition 2.9. Tautology: ϕ is a tautology $\iff \forall v, v \models \phi$

Example 2.10.

$$\phi = \neg(p \vee (\neg p))$$

From example 2.4, we have

$$\begin{aligned} \forall v, v \models (p \vee (\neg p)) \\ \rightarrow \forall v, v \not\models \neg(p \vee (\neg p)) \end{aligned}$$

Thus ϕ is NOT satisfiable.

2.4 Satisfiability of a set of formula

Set Satisfaction

Definition 2.11. Let $\Gamma \subseteq FORM$,

$$v \models \Gamma \iff \forall \phi \in \Gamma, v \models \phi$$

Set Satisfiability

Definition 2.12. Let $\Gamma \subseteq FORM$, Γ is satisfiable

$$\iff \exists v, \forall \phi \in \Gamma, v \models \phi$$

Example 2.13. $\Gamma = \emptyset$, Γ is satisfiable.

Proof. Consider $v : P \mapsto \{T, F\}$, $v(p) = T \forall p \in P$, $\forall \phi \in \Gamma, v \models \phi$ is vacuously true. □

Example 2.14. $\Gamma = FORM$, Γ is NOT satisfiable.

Proof. Suppose otherwise,

$$\begin{aligned} \exists v, \forall \phi \in \Gamma, v \models \phi \\ \forall \phi \in \Gamma, (\neg \phi) \in \Gamma \\ v \models (\neg \phi) \end{aligned}$$

A contradiction. □

2.4.1 Relevance

Definition 2.15. Define set $OCCUR(\phi) \subseteq P$, which contains propositions appear in ϕ . Inductively defined as

$$\begin{aligned} OCCUR(\phi) &= \{p\} \iff \phi \equiv p, p \in P \\ OCCUR(\neg \phi) &= OCCUR(\phi) \\ OCCUR(\phi_1 \vee \phi_2) &= OCCUR(\phi_1) \cup OCCUR(\phi_2) \end{aligned}$$

Lemma 2.16. Let $\phi \in FORM$, let v_1, v_2 be valuations over P .

$$\forall p \in OCCUR(\phi), v_1(p) = v_2(p) \rightarrow (v_1 \models \phi \rightarrow v_2 \models \phi)$$

v_1, v_2 can be different valuations over $p' \notin OCCUR(\phi)$

Proof. Let $p \in P$, □

2.4.2 Logical consequence

Semantic logical consequence:

Definition 2.17.

$$\Gamma \models \phi \iff \forall v \models \Gamma, v \models \phi$$

Syntactic logical consequence: (logical derivability)

Definition 2.18. A formula ϕ is a syntactic consequence of Γ if there exists a formal proof (a finite sequence of formulas) ending in ϕ , where each step is either a premise from Γ , an axiom, or follows from previous steps via an inference rule (like Modus Ponens).

Material Implication ($\rightarrow$) vs. Logical Consequence ($\models$)

- Material implication is a connector within a formula, $p \rightarrow q \equiv \neg(p \vee q)$, and can therefore be eval to true or false in a single model.
- Logical consequence is a relation between formulas (meta-logic), it is a statement about all models. $\{p, p \rightarrow q\} \models q$

2.5 Modelling Computations as Propositional Logic

Computation problems can be considered as membership check of an encoded problem instance to a language.

$$L = \{p \mid \langle M, p \rangle\}$$

Here instead of arbitrary encoding, we consider problem encoding with propositional logics only, and solve with SAT solvers by deciding whether the set of formula we form from any instance of the problem is satisfiable. That is

$$Encode : p \mapsto FORM, \forall p, \exists v \models Encode(p) \iff p \in L$$

The rules of such modelling include:

- Propositional logics only
- The final expression must one well formed fomula
- for problem of size n , the size of the formula ϕ must be in $O(poly(n))$

Note that there can be different definition on the size of the

- problem: in unary, binary etc;
- Formula: Whether by the number of nodes in the syntax tree or length of the string **Anything else?**

Example 2.19. Graph coloring problem: given a undirected graph $G = (V, E)$, whether there exists a k coloring of all vertices such that no neighbouring vertices have the same color.

Consider the following construction of a proposition set P :

$$P = \{p_{v,i} \mid v \in V, i \in [1, k]\}$$

The valuation of proposition $p_{v,i}$ conceptually represents vertex v is assigned color i .

We can formulate constraints as well formed formula, and use conjunction to connect the formula to model satisfaction of multiple constraints.

Corollary 2.20. $\forall \phi_1, \phi_2 \in FORM, (\phi_1 \wedge \phi_2) \in FORM$

First let us list down the constraints of this problems, including implicit ones in plain words

1. Neighbouring vertices have different colors.
2. All vertices as assigned with exactly one color.

1. ϕ_{nbr}

First lets formula the constraint between any 2 vertices,

$$\forall (\alpha, \beta) \in E, c(v_\alpha) \neq c(v_\beta)$$

And from our proposition set, $c(v_\gamma) = i \iff \exists v, v \models p_{\gamma,i}$. As such we have

$$\phi_{\alpha,\beta,i} = \neg(p_{\alpha,i} \wedge p_{\beta,i})$$

This models vertex α and vertex β are not colored with color i at the same time. For every color this needs to hold, thus we have

$$\phi_{nbr(\alpha,\beta)} = \bigwedge_{i \in [1,k]} \phi_{\alpha,\beta,i}$$

For every pair of neighbouring vertices this also needs to hold. Therefore we have

$$\phi_{nbr} = \bigwedge_{(\alpha,\beta) \in E} \phi_{nbr(\alpha,\beta)}$$

2. $\phi_{\text{exactly-1}}$

To model exactly 1 coloring on each vertex, we can formulate $\phi_{\geq 1}$ and $\phi_{\leq 1}$ first and use conjunction to connect them. Given our propositions, to make sure vertex v is at least assign a color ($\phi_{v,\geq 1}$),

$$\phi_{v,\geq 1} = \bigvee_{i \in [1,k]} p_{v,i}$$

This needs to hold for every vertex, thus we have

$$\phi_{\geq 1} = \bigwedge_{v \in V} \phi_{v,\geq 1}$$

Then we need to model at most 1, (or less than 2) coloring on each vertex. Vertex v is not assignment with both color i and j can be formulated as

$$\phi_{v,i,j,\leq 1} = \neg(p_{v,i} \wedge p_{v,j})$$

This needs to hold for every distinct pair of colors, thus

$$\phi_{v,\leq 1} = \bigwedge_{i \neq j \in [1,k]} \phi_{v,i,j,\leq 1}$$

For all vertices

$$\phi_{\leq 1} = \bigwedge_{v \in V} \phi_{v,\leq 1}$$

Finally we have

$$\phi_{\text{exactly-1}} = \phi_{\leq 1} \wedge \phi_{\geq 1}$$

And the overall formula for an instance of $G = (V, E)$ becomes

$$\phi = \phi_{\text{nbr}} \wedge \phi_{\text{exactly-1}}$$

Similar to reduction, we need also now to show that

$$\exists v : P \mapsto \{T, F\}, v \models \phi \iff \langle G = (V, E), k \rangle \in L_{\text{color}}$$

TODO

Introduce propositional variables that encode atomic choices, then add constraints that enforce the combinatorial structure of the problem.

1. Hamiltonian Path

Instance. Graph $G = (V, E)$ with $|V| = n$.

Variables. For every vertex $v \in V$ and position $i \in \{1, \dots, n\}$:

$$X_{v,i} \quad \text{“vertex } v \text{ is at position } i \text{ in the path.”}$$

Constraints.

1. **Each position has exactly one vertex:**

$$\bigvee_{v \in V} X_{v,i} \quad \text{and pairwise exclusion.}$$

2. **Each vertex appears exactly once:**

$$\bigvee_{i=1}^n X_{v,i} \quad \text{and pairwise exclusion.}$$

3. **Adjacency constraint:** If $(u, v) \notin E$, then for all i :

$$\neg X_{u,i} \vee \neg X_{v,i+1}.$$

Satisfiable iff G has a Hamiltonian path.

2. Clique of Size k

Instance. Graph $G = (V, E)$.

Variables. For each vertex $v \in V$:

X_v “vertex v is selected.”

Constraints.

1. **At least k vertices selected:** Encode using a cardinality constraint.
2. **Selected vertices form a clique:** If $(u, v) \notin E$, then

$$\neg X_u \vee \neg X_v.$$

Satisfiable iff there exists a clique of size $\geq k$.

3. 3-Dimensional Matching

Instance. Sets X, Y, Z and triples $T \subseteq X \times Y \times Z$.

Variables. For each triple $t \in T$:

X_t “triple t is chosen.”

Constraints.

1. **No element used twice:**
If triples t, t' share an element, then
2. **Select exactly k triples:** Encode cardinality constraint.

$$\neg X_t \vee \neg X_{t'}.$$

Satisfiable iff there is a matching of size k .

4. Subset Sum

Instance. Numbers $a_1, \dots, a_n$, target B .

Variables.

X_i “ a_i is selected.”

Constraints. We encode:

$$\sum_{i=1}^n a_i X_i = B.$$

This is reduced to propositional logic by building a binary addition circuit:

- Introduce variables for each bit of partial sums.
- Encode full-adder constraints using propositional clauses.

Satisfiable iff there exists a subset summing to B .

5. Exact Cover

Instance. Universe U , collection $\mathcal{S} = \{S_1, \dots, S_m\}$.

Variables.

X_i “set S_i is selected.”

Constraints. For every element $u \in U$:

1. **At least one covering set:**

$$\bigvee_{u \in S_i} X_i$$

2. **At most one covering set:** If $u \in S_i \cap S_j$ and $i \neq j$:

$$\neg X_i \vee \neg X_j.$$

Satisfiable iff an exact cover exists.

6. Scheduling with Precedence Constraints

Instance. Tasks $T_1, \dots, T_n$, time slots $1, \dots, m$.

Variables.

$X_{i,t}$ “task i scheduled at time t .”

Constraints.

1. Each task scheduled exactly once.
2. At most one task per time slot.
3. If T_i must precede T_j :

$$X_{i,t} \rightarrow \bigvee_{t' > t} X_{j,t'}.$$

2.6 Encoding a Full Adder in Pure Propositional Logic

A full adder takes three input bits:

$$x, y, c_{in}$$

and produces two outputs:

$$s \quad (\text{sum bit}), \quad c_{out} \quad (\text{carry bit})$$

Semantically:

$$\begin{aligned} s &= x \oplus y \oplus c_{in} \\ c_{out} &= (x \wedge y) \vee (x \wedge c_{in}) \vee (y \wedge c_{in}) \end{aligned}$$

We now encode this purely in propositional logic using only $\wedge, \vee, \neg$ (or $\rightarrow, \perp$ if preferred).

1. Encoding the Sum Bit

The sum is true iff an odd number of inputs are true.

Truth table characterization:

$$s \leftrightarrow (x \wedge \neg y \wedge \neg c_{in}) \vee (\neg x \wedge y \wedge \neg c_{in}) \vee (\neg x \wedge \neg y \wedge c_{in}) \vee (x \wedge y \wedge c_{in})$$

Thus we encode:

$$(s \rightarrow \Phi_s) \wedge (\Phi_s \rightarrow s)$$

where Φ_s is the disjunction above.

2. Encoding the Carry Bit

Carry is true iff at least two inputs are true:

$$c_{out} \leftrightarrow (x \wedge y) \vee (x \wedge c_{in}) \vee (y \wedge c_{in})$$

Again encode as:

$$(c_{out} \rightarrow \Phi_c) \wedge (\Phi_c \rightarrow c_{out})$$

3. CNF Encoding (SAT-friendly)

To encode for SAT, we introduce clauses.

Carry Constraints

$$\neg c_{out} \vee x \vee y$$

$$\neg c_{out} \vee x \vee c_{in}$$

$$\neg c_{out} \vee y \vee c_{in}$$

$$\neg x \vee \neg y \vee c_{out}$$

$$\neg x \vee \neg c_{in} \vee c_{out}$$

$$\neg y \vee \neg c_{in} \vee c_{out}$$

Sum Constraints (XOR Form) We can encode $s = x \oplus y \oplus c_{in}$ using clauses:

$$(x \vee y \vee c_{in} \vee \neg s)$$

$$(\neg x \vee \neg y \vee c_{in} \vee \neg s)$$

$$(\neg x \vee y \vee \neg c_{in} \vee \neg s)$$

$$(x \vee \neg y \vee \neg c_{in} \vee \neg s)$$

$$(\neg x \vee \neg y \vee \neg c_{in} \vee s)$$

$$(x \vee y \vee \neg c_{in} \vee s)$$

$$(x \vee \neg y \vee c_{in} \vee s)$$

$$(\neg x \vee y \vee c_{in} \vee s)$$

These clauses exactly encode the XOR behavior.

4. Circuit-Style Encoding (Cleaner)

Instead of expanding XOR directly, introduce auxiliary variables:

$$t = x \oplus y$$

$$s = t \oplus c_{in}$$

Each XOR can be encoded with 4 clauses:

$$(t \leftrightarrow x \oplus y)$$

which in CNF becomes:

$$(\neg x \vee \neg y \vee \neg t) \wedge (x \vee y \vee \neg t) \wedge (x \vee \neg y \vee t) \wedge (\neg x \vee y \vee t)$$

This keeps formula size linear.

5. Why This Matters

Full adders are the atomic building blocks of:

- Binary addition circuits
- Subset Sum encodings
- Integer programming to SAT reductions
- Cook–Levin tableau arithmetic

Any arithmetic constraint can be reduced to propositional logic by composing full adders.

Conceptual Insight

Arithmetic $\longrightarrow$ Circuits $\longrightarrow$ Propositional Logic

This translation is what makes SAT complete for NP: even numeric computation can be flattened into Boolean structure.

General Pattern

Almost all NP problems can be modelled by:

1. Encoding choices with propositional variables.
2. Enforcing structural constraints with local clauses.
3. Encoding counting via cardinality circuits if needed.

This modelling perspective is precisely what makes SAT solvers universal engines for combinatorial search.

3 Compactness Theorem for Propositional Logics

Definition 3.1. *Axiom of Choice: formally*

$$\forall X, \emptyset \in X \vee \exists f, \text{dom}(f) = X \wedge \forall A \in X, f(A) \in A$$

A choice function (or selection) f defined on a collection of nonempty sets X , such that for every set A in the collection X , $f(A)$ is an element in A . That is, Axiom—For any set X of nonempty sets, there exists a choice function f that is defined on X and maps each set of X to an element of that set.

Theorem 3.2. *If P is finite, FORM over P is finite.*

Theorem 3.3. *If P is countable, FORM over P is countable.*

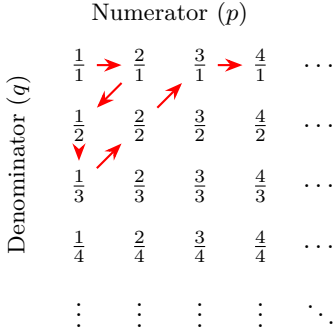
Proof. Given $\forall \phi \in \text{FORM}, \phi$ is finitely long, this theorem is another way of stating: The language formed from countable alphabet $P \cup C$ with the inductive construction rules, is countable. We can prove a stronger theorem first. $\square$

Theorem 3.4. *All languages formed from a countable alphabet is countable.*

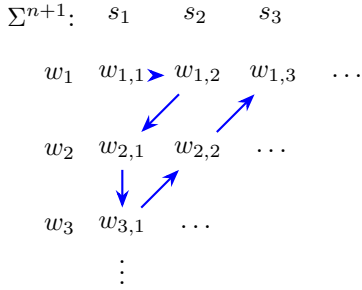
Proof. Consider $L = \Sigma^*$, we will prove that Σ is countable $\rightarrow L$ is countable. We use the same diagonal proof for the countability of $\mathbb{Q}$ to show an enumeration of all strings formable from a countable alphabet. To formalize, we use induction to prove that $\forall i \geq 0, \Sigma^i$ is countable
Base case: $\Sigma^1 = \Sigma$, countable as defined.

Inductive hypothesis: Σ^n is countable: we enumerate all strings in Σ^n , and use the same diagonal proof to show the concatenation of one symbol from the countable alphabet to every string Σ^{n+1} is also countable. Thus we can give

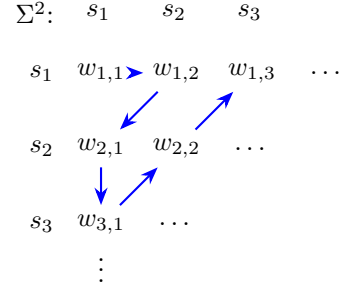
Rational Numbers ($\mathbb{Q}^+$)



Countable Language (Σ^{n+1})



Countable Language (Σ^2)



Countable Language (Σ^*)

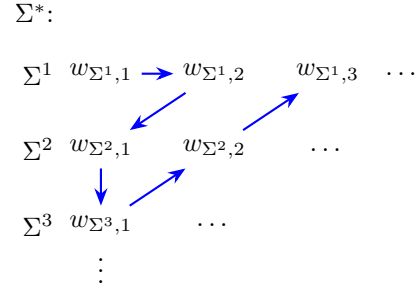


Figure 1: Comparison of diagonal enumeration for numbers vs. strings.

an enumeration of all Σ^i and use the diagonal argument again to show the existence of an enumeration of Σ^* . Alternatively, Given

$$\Sigma^* = \bigcup_{i \geq 0} \Sigma^i$$

From the Axiom of Choice, which implies the union of countably many countable set is countable, Σ^* is countable. Since $\forall L$ over $\Sigma, L \subseteq \Sigma^*, \Sigma^*$ is countable, all languages over countable Σ is countable. $\square$

Definition 3.5 (Finitely Satisfiable). *A set of formulas Γ is said to be **finitely satisfiable** if and only if for all finite subsets $\Gamma_0 \subseteq \Gamma$, Γ_0 is satisfiable.*

Theorem 3.6 (Compactness Theorem).

$$\Gamma \text{ is satisfiable} \iff \Gamma \text{ is finitely satisfiable}$$

Proof. Consider a special case where Γ is “ T -good”, defined as:

$$\forall p \in P, (p \in \Gamma \vee \neg p \in \Gamma)$$

Claim: If Γ is T -good, then Γ is finitely satisfiable $\implies \Gamma$ is satisfiable.

Suppose Γ is T -good and finitely satisfiable. We define a candidate valuation v_L such that for all $p \in P$:

$$v_L(p) = \begin{cases} T & \text{if } p \in \Gamma \\ F & \text{if } \neg p \in \Gamma \end{cases}$$

First, we prove that for every $p \in P$, **exactly one** of $\{p, \neg p\}$ is in Γ . By the T -good property, at least one is in Γ . Suppose both $\{p, \neg p\} \subseteq \Gamma$. Consider the finite subset:

$$\Gamma' = \{p, \neg p\}$$

Since Γ' is a finite subset of a finitely satisfiable set, it must be satisfiable. However, $\neg \exists v$ such that $v \models \{p, \neg p\}$, which is a contradiction. Thus, v_L is well-defined.

Now we show $v_L \models \Gamma$. Suppose $\exists \phi \in \Gamma$ such that $v_L \not\models \phi$. Define the following finite sets based on the atoms occurring in ϕ :

$$\begin{aligned}\Gamma_\phi^+ &= \{p \mid p \in \text{OCCUR}(\phi) \wedge v_L(p) = T\} \\ \Gamma_\phi^- &= \{\neg p \mid p \in \text{OCCUR}(\phi) \wedge v_L(p) = F\} \\ \Gamma_\phi &= \{\phi\} \cup \Gamma_\phi^+ \cup \Gamma_\phi^-\end{aligned}$$

Since Γ_ϕ is a finite subset of Γ , it must be satisfiable by some valuation v . For this v :

- $\forall p$ such that $p \in \Gamma_\phi^+$, $v(p) = T$.
- $\forall \neg p$ such that $\neg p \in \Gamma_\phi^-$, $v(\neg p) = T \implies v(p) = F$.

By construction, v and v_L agree on all atoms in $\text{OCCUR}(\phi)$. By the **Relevance Lemma**, $v(\phi) = v_L(\phi)$. Since $v \models \Gamma_\phi$, we have $v \models \phi$. This implies $v_L \models \phi$, contradicting our assumption $v_L \not\models \phi$. Therefore, $\forall \phi \in \Gamma, v_L \models \phi$, which implies $v_L \models \Gamma$, Γ is satisfiable.

Now we will generalize this to all Γ which may not be ‘T-good’, by showing that we can construct a ‘T-good’ finitely satisfiable set Δ , from any finitely satisfiable Γ , and therefore show that $\Gamma SAT \iff \Delta SAT$.

We will only prove the case for countable P now. Let an enumeration of $p \in P$ be $\{p_1, p_2, \dots\}$

Define finite subset Γ_i of Γ as such:

$$\begin{aligned}\Gamma_0 &= \Gamma \\ \Gamma_{i+1} &= \begin{cases} \Gamma_i \cup \{p_i\} & \text{if } \Gamma_i \cup \{p_i\} \text{ is finitely satisfiable} \\ \Gamma_i \cup \{\neg(p_i)\} & \text{Otherwise} \end{cases}\end{aligned}$$

Here we need to prove that $\forall i$, between $\Gamma_i \cup \{p_i\}$ and $\Gamma_i \cup \{\neg(p_i)\}$ at least one case is also finitely satisfiable.

Theorem 3.7. *Let Γ be finitely satisfiable set, and ϕ be a formula, then $\Gamma \cup \{\phi\}$ is finitely satisfiable or $\Gamma \cup \{\neg(\phi)\}$ is finitely satisfiable.*

Proof. We prove the contrapositive of this statement: if $\Gamma \cup \{\phi\}$ and $\Gamma \cup \{\neg(\phi)\}$ are both not finitely satisfiable, Γ is not finitely satisfiable. Given both are not finitely satisfiable, there exist finite subsets $\Gamma_1, \Gamma_2 \subseteq \Gamma$ such that $\Gamma_1 \cup \{\phi\}$ and $\Gamma_2 \cup \{\neg(\phi)\}$ are not satisfiable. Therefore for all valuations, it must not model either Γ_1 or $\{\phi\}$, and not model either Γ_2 or $\{\neg(\phi)\}$. Denote the set of valuations that models formula ψ as $models(\psi)$, the set of valuations that models set of formula Σ as $models(\Sigma)$

$$\begin{aligned}models(\Gamma_1) \cap models(\phi) &= \emptyset \\ \rightarrow \forall v \in models(\Gamma_1), v \not\models \phi \rightarrow v \models (\neg\phi) \\ \rightarrow models(\Gamma_1) &\subseteq models((\neg\phi)) \\ models(\Gamma_2) \cap models((\neg\phi)) &= \emptyset \\ \rightarrow \forall v \in models(\Gamma_2), v \not\models (\neg\phi) \rightarrow v \models (\phi) \\ \rightarrow models(\Gamma_2) &\subseteq models(\phi)\end{aligned}$$

Now consider the set of valuations that models $\Gamma_1 \cup \Gamma_2$

$$\begin{aligned}models(\Gamma_1 \cup \Gamma_2) &= models(\Gamma_1) \cap models(\Gamma_2) \\ (models(\Gamma_1) \cap models(\Gamma_2)) &\subseteq (models(\phi) \cap models((\neg\phi))) = \emptyset\end{aligned}$$

Since $\Gamma_1 \cup \Gamma_2$ is a finite subset of Γ that is not satisfiable, Γ is not finitely satisfiable. □

Now prove that

Lemma 3.8. $\forall i, \Gamma_i$ is finitely satisfiable.

Proof. Base case:

$$\Gamma_0 = \Gamma \text{ is finitely satisfiable by definition}$$

Inductive hypothesis: Suppose

$$\Gamma_n \text{ is finitely satisfiable}$$

By Lemma 3.7, $\Gamma_n \cup \{p_n\}$ and $\Gamma_n \cup \{\neg p_n\}$ there must be at least one that is finitely satisfiable. $\rightarrow \Gamma_{n+1}$ is therefore finitely satisfiable. $\square$

Next we will show that

Lemma 3.9. $\Gamma_U = \bigcup_{i \geq 0} \Gamma_i$ is finitely satisfiable.

Proof. Consider finite set $\Gamma' \subseteq \Gamma_U$, since Γ' is finite, and $\forall k \geq 0, \Gamma_k \subseteq \Gamma_{k+1}$ by definition, $\exists j \geq 0, \Gamma' \subseteq \Gamma_j = \bigcup_{i \geq 0} \Gamma_i$. By Lemma 3.8, Γ_j is satisfiable $\rightarrow \Gamma'$ is finitely satisfiable. $\square$

Lemma 3.10. Γ_U is satisfiable.

Proof. From the construction, $\forall i \geq 0$ at least one of p_i and $(\neg p_i)$ is in $\bigcup_{i \geq 0} \Gamma_i$. Since $\bigcup_{i \geq 0} \Gamma_i$ is finitely satisfiable, $\forall i \geq 0, p_i$ and $(\neg p_i)$ cannot both be in $\bigcup_{i \geq 0} \Gamma_i$. Otherwise $\{p_i\} \cup \{(\neg p_i)\} \subseteq \bigcup_{i \geq 0} \Gamma_i$ is not satisfiable, contradiction. $\square$

Now we have proven that the construction proposed from a finitely satisfiable formula set Γ to $\bigcup_{i \geq 0} \Gamma_i$ is also finitely satisfiable. Now to complete the proof of the compactness theorem, we need to prove that $\exists v, v \models \Gamma$.

Proof. From Lemma 3.10, $\forall i \geq 0, (p_i \in \Gamma_U) \oplus ((\neg p_i) \in \Gamma_U)$, $\exists v, v \models \Gamma_U$

$$v(p) : P \mapsto \{T, F\} = \begin{cases} T, & \text{if } p \in \Gamma_U \\ F, & \text{if } (\neg p) \in \Gamma_U \end{cases}$$

Define the literal set $Literals = \{p_i, (\neg p_i) \mid i \geq 0\}$, and function

$$\tau(p) : P \mapsto Literals = \begin{cases} p, & \text{if } p \in \Gamma_U \\ (\neg p), & \text{if } (\neg p) \in \Gamma_U \end{cases}$$

Lemma 3.11.

$$v \models \Gamma_U \rightarrow v \models \tau(\Gamma)$$

Let $\phi \in \Gamma$, Define literal set $X = \{\tau(p) \mid p \in OCCUR(\phi)\}$ where X is a set containing single proposition literals. By definition we have

$$X \subseteq \Gamma_U \rightarrow v \models X \rightarrow \forall \psi \in X, v(\psi) = T$$

and $OCCUR(X) = OCCUR(\phi)$ Since both X is finite, $X \cup \{\phi\}$ is a finite subset to Γ_U , thus satisfiable. $\rightarrow \exists v', v' \models X \cup \{\phi\} \rightarrow v' \models X, v' \models \phi \rightarrow \forall \psi \in X, v'(\psi) = T$.

$$\forall p \in OCCUR(\phi), v(p) = v'(p), v' \models \phi \rightarrow v \models \phi$$

Therefore, the valuation that models enlarged construction Γ_U also models Γ , thus Γ is satisfiable. $\square$

$\square$

3.1 Application of the Compactness theorem for proposition logics

We have already shown that we can use propositional logic to model computational problems, the compactness theorem allows us to extend problem modelling to instances of infinite size. Recall the k-colorable problem, where we used propositional logic to model any finite size instance.

Theorem 3.12. A graph is k-colorable iff every finite subgraph is k-colorable.

Definition 3.13 (Infinite graph). $G = (V, E)$ is an infinite graph if V is infinite.

Definition 3.14 (Subgraph). $G' = (V', E')$ is a subgraph to G if $V' \subseteq V, E' \subseteq E$

Definition 3.15 (Induced subgraph). $G' = (V', E')$ is an induced subgraph to G if G' is a subgraph to G , and $\{u, v\} \in E' \iff \{u, v\} \in E \wedge u, v \in V'$

Proof. Since we have formulated any finite instance of k -colorable graph as propositional logic formula, we can define infinite set

$$\Gamma_G = \{\phi_{G'}^{k\text{-colorable}} \mid G' \text{ is finite subgraph of } G\}$$

We will first prove that

Lemma 3.16. G is finitely k -colorable $\rightarrow \Gamma_G$ is finitely satisfiable.

Proof. We will reuse the proposition definition from last chapter, where $|V|$ and thus P is countable

$$P = \{p_{v,i} \mid v \in V, i \in [1, k]\}$$

For each instance on $G = (V, E)$ and k colors, we formulate as such to enforce constraints on edges (No same color on adjacent vertices), and vertices (exactly 1 coloring on each vertex).

$$\phi = \phi^{nbr} \wedge \phi^{exactly-1\text{-color}}$$

Consider the infinite graph $G = (V, E)$

$$\begin{aligned} \Gamma_G &= \Gamma_G^{nbr} \cup \Gamma_G^{exactly-1\text{-color}} \\ \Gamma_G^{exactly-1\text{-color}} &= \{\phi_v^{exactly-1\text{-color}} \mid v \in V\} \\ \phi_v^{exactly-1\text{-color}} &= \left(\bigvee_{i \in [1, k]} p_{v,i} \right) \wedge \left(\bigwedge_{i \neq j \in [1, k]} (p_{v,i} \rightarrow p_{v,j}) \right) \\ \Gamma_G^{nbr} &= \{\phi_{\{u,v\}}^{nbr} \mid \{u, v\} \in E\} \\ \phi_{\{u,v\}}^{nbr} &= \bigwedge_{i \in [1, k]} (p_{u,i} \rightarrow p_{v,i}) \end{aligned}$$

Suppose the contrary, where $G = (V, E)$ is finitely k -colorable, Γ_G is not finitely satisfiable, $\exists$ finite subset $\Gamma_0 \subseteq \Gamma_G, \Gamma_0$ is not satisfiable. Let $G_0 = (V_0, E_0)$ be a finite subgraph of G , such that

$$\begin{aligned} V_0 &= \{v \mid v \in V, \phi_v^{exactly-1\text{-color}} \in \Gamma_0\} \cup \{u \mid u \in V, \exists u', \phi_{\{u,u'\}}^{nbr} \in \Gamma_0\} \\ E_0 &= \{\{u, v\} \mid \{u, v\} \in E, u, v \in V_0\} \end{aligned}$$

Here we are formulating an instance of finite k -colorable induced subgraph of G from finite subset Γ_0 . Now we need to show that G_0 is not k -colorable to reach the contradiction, with another contradiction.

Suppose G_0 is k -colorable, let C_0 be a k -coloring of G_0 , define valuation

$$v_0(p_{u,i}) = \begin{cases} T, & \text{if } C_0(u) = i \\ F, & \text{if } C_0(u) \neq i \end{cases}$$

Let

$$\Gamma'_0 = \{\phi_v^{exactly-1\text{-color}} \mid v \in V_0\} \cup \{\phi_{\{u,v\}}^{nbr} \mid \{u, v\} \in E_0\}$$

Consider

$$\Gamma_0 \subseteq \Gamma'_0$$

Since Γ'_0 is the same construction from k -colorability to formula, G_0 is k -colorable, we have

$$v_0 \models \Gamma'_0$$

By relevance lemma

$$v_0 \models \Gamma_0$$

Contradicting to our initial assumption that Γ_0 is NOT satisfiable. $\square$

By Compactness theorem, Γ_G is finitely satisfiable $\rightarrow \Gamma_G$ is satisfiable.

Lemma 3.17. Γ_G is satisfiable $\rightarrow G$ is k -colorable

Proof. Γ_G is satisfiable $\rightarrow \exists v, v \models \Gamma_G$. We can construct a coloring C from v similarly to how we constructed v_0 from C_0 from previous proof:

$$C(v) = i, \text{ if } v(p_{v,i}) = T$$

By construction the coloring is valid, detailed technicality is omitted. □

□

3.2 Proof of König's Lemma via Compactness

König's Lemma states that every infinite, finitely branching tree contains an infinite path. We can prove this elegantly using the Compactness Theorem for Propositional Logic, which states that a set of formulas Γ is satisfiable if and only if every finite subset of Γ is satisfiable.

Theorem 3.18 (König's Lemma). *Let T be a tree with an infinite number of vertices such that every vertex has a finite degree (finitely branching). Then T contains at least one infinite path starting from the root.*

Proof. To prove this using propositional logic, we define a set of propositional variables and constraints that force the existence of an infinite path. **1. Define Propositional Variables:** For every vertex $v \in T$, let P_v be a propositional variable. The intended meaning is:

$$P_v \text{ is true } \iff v \text{ is part of the infinite path.}$$

2. Construct the Set of Formulas Γ : We include the following requirements in our set Γ :

- **Existence at each level:** For each natural number n , at least one vertex at depth n must be on the path.

$$\bigvee_{v \in \text{Level}(n)} P_v$$

(Note: This is a finite disjunction because the tree is finitely branching.)

- **Uniqueness per level:** No two distinct vertices u, v at the same level can both be on the path.

$$\neg(P_u \wedge P_v) \quad \text{for } u, v \in \text{Level}(n), u \neq v$$

- **Structure (Parental Link):** If a vertex v (at depth > 0) is on the path, its unique parent u must also be on the path.

$$P_v \rightarrow P_u$$

3. Apply Compactness: Consider any finite subset $\Gamma_0 \subseteq \Gamma$. Let N be the maximum depth of any vertex mentioned in Γ_0 . Since the tree is infinite and finitely branching, there must be at least one vertex v_{max} at depth N . We can satisfy Γ_0 by picking v_{max} and its ancestors back to the root, setting those variables to *true* and all others to *false*. Since every finite subset Γ_0 is satisfiable, the Compactness Theorem implies that the entire set Γ is satisfiable. **4.**

Conclusion: Let $\mathcal{I}$ be a satisfying assignment for Γ . The set of vertices $V_{path} = \{v \in T \mid \mathcal{I}(P_v) = \text{true}\}$ defines a sequence of vertices starting from the root where each level has exactly one vertex, and each vertex is connected to its parent. This sequence forms an infinite path. □

4 Proof System

4.1 Hilbert Style Proof System

Four axiom schema of the hilberts style proof system. Axiom schema means that the ϕ, ψ, ρ in these schema are placeholders that can be replaced by any valid formula

1. Axiom 1: Simplification

$$\frac{}{p \rightarrow (q \rightarrow p)}$$

2. Axiom 2: Distribution

$$\frac{}{(p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))}$$

3. Axiom 3: Double Negation Elimination (DNE)

$$\frac{}{((p \rightarrow \perp) \rightarrow \perp) \rightarrow p}$$

4. Rule of Inference: Modus Ponens

$$\frac{p \rightarrow q \quad p}{q}$$

The horizontal lines are judgement lines, which means that whats below can be inferred from whats above. The first 3 rules has no assumptions or premises, meaning they are ways we can introduce arbitrary claims or assumption into our proofs.

Definition 4.1 (Proof). *A proof of the consequence of ϕ from Γ is a finite sequence of formula $c_1, c_2, \dots, c_k$ where $\forall i \leq k, c_i \in \Gamma$ or, c_i is obtained using one of the 4 axioms:*

- $\exists j, l < i, c_j \equiv c_l \rightarrow c_i$
- $\exists \phi, \psi, c_i \equiv \phi \rightarrow (\psi \rightarrow \phi)$
- $\exists \phi, \psi, \rho, c_i \equiv [\phi \rightarrow (\psi \rightarrow \rho)] \rightarrow [(\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \rho)]$
- $\exists \phi, c_i \equiv ((\phi \rightarrow \perp) \rightarrow \perp) \rightarrow \phi$

Definition 4.2 (Derivable). *ϕ is derivable from Γ if there is a proof in HSPS of " ϕ is a consequence of Γ "*

$$\Gamma \vdash \phi$$

Example 4.3.

$$\Gamma = \{p, p \rightarrow q\}, \quad \phi = q$$

Proof. We can construct proof sequence:

$c_1 = p$	$(p \in \Gamma)$
$c_2 = p \rightarrow q$	$(p \rightarrow q \in \Gamma)$
$c_3 = q$	(Modus Ponens on c_1, c_2)

□

Sometimes HSPS requires us to creatively introduce new hypothesis to assist in the proof.

Example 4.4.

$$\Gamma = \{\perp\}, \phi = p$$

Intuitively this does not sound provable, just from $\perp$, but axioms that require no premises allows us to introduce helper hypothesis. In this case $(p \rightarrow \perp)$

Proof.

$c_1 = \perp$	$(\perp \in \Gamma)$
$c_2 = \perp \rightarrow [(p \rightarrow \perp) \rightarrow \perp]$	(Axiom 1: Simplification)
$c_3 = (p \rightarrow \perp) \rightarrow \perp$	(Modus Ponens on $\perp, (p \rightarrow \perp)$)
$c_4 = [(p \rightarrow \perp) \rightarrow \perp] \rightarrow p$	(Axiom 3: Double negation)
$c_5 = p$	(Modus Ponens on $c_3 \equiv [(p \rightarrow \perp) \rightarrow \perp], c_4 \equiv [(p \rightarrow \perp) \rightarrow \perp] \rightarrow p$)

□

As seen from this example, HSPS can get convoluted and unintuitive for relatively straightforward consequences.

1. Self-Implication: $\vdash p \rightarrow p$

Idea. We must simulate identity using only distribution.

1. $p \rightarrow ((p \rightarrow p) \rightarrow p)$ (Axiom 1)
2. $p \rightarrow (p \rightarrow p)$ (Axiom 1)
3. $(p \rightarrow ((p \rightarrow p) \rightarrow p)) \rightarrow ((p \rightarrow (p \rightarrow p)) \rightarrow (p \rightarrow p))$ (Axiom 2)
4. $(p \rightarrow (p \rightarrow p)) \rightarrow (p \rightarrow p)$ (MP 1,3)
5. $p \rightarrow p$ (MP 2,4)

Even the trivial tautology requires a nontrivial derivation.

2. Contraposition: $\vdash (p \rightarrow q) \rightarrow ((q \rightarrow \perp) \rightarrow (p \rightarrow \perp))$

Idea. Assume $p \rightarrow q$ and $q \rightarrow \perp$; derive $p \rightarrow \perp$ via Axiom 2.

1. $(p \rightarrow (q \rightarrow \perp)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow \perp))$ (Axiom 2)
2. $(q \rightarrow \perp) \rightarrow (p \rightarrow (q \rightarrow \perp))$ (Axiom 1)
3. $(p \rightarrow q) \rightarrow ((q \rightarrow \perp) \rightarrow (p \rightarrow \perp))$

The last line follows by two applications of Modus Ponens using (1) and (2).
This shows implication reversal must be encoded indirectly.

3. Peirce's Law: $\vdash ((p \rightarrow q) \rightarrow p) \rightarrow p$

This formula characterizes classical logic.

Sketch.

1. $((p \rightarrow q) \rightarrow p) \rightarrow (((p \rightarrow q) \rightarrow p) \rightarrow ((p \rightarrow q) \rightarrow p))$ (Ax1)
2. Use Axiom 2 repeatedly to simulate assumption chaining.
3. Derive $((p \rightarrow q) \rightarrow p) \rightarrow \perp \rightarrow \perp$.
4. Apply Axiom 3 (DNE) to conclude $((p \rightarrow q) \rightarrow p) \rightarrow p$.

The essential trick: Peirce's law is obtained by embedding the formula into a double-negation context and applying Axiom 3.

This is far from syntactically obvious.

4. Explosion: $\vdash \perp \rightarrow p$

In this system $\perp$ is primitive via $p \rightarrow \perp$.

Idea. Show that from $\perp$ we can derive anything.

1. $p \rightarrow (\perp \rightarrow p)$ (Ax1)
2. $\perp \rightarrow (p \rightarrow \perp)$ (Ax1)
3. $(\perp \rightarrow (p \rightarrow \perp)) \rightarrow ((\perp \rightarrow p) \rightarrow (\perp \rightarrow \perp))$ (Ax2)
4. Derive $\perp \rightarrow \perp$ using the $p \rightarrow p$ pattern.
5. Conclude $\perp \rightarrow p$.

Notice how even explosion must be simulated through distribution.

5. Double Negation Introduction: $\vdash p \rightarrow ((p \rightarrow \perp) \rightarrow \perp)$

Although Axiom 3 gives elimination, introduction must be derived.

1. $(p \rightarrow \perp) \rightarrow (p \rightarrow (p \rightarrow \perp))$ (Ax1)
2. Use Axiom 2 to rearrange implication nesting.
3. Conclude $p \rightarrow ((p \rightarrow \perp) \rightarrow \perp)$.

This illustrates asymmetry between DNE and its converse.

4.2 Metatheorems

4.2.1 Monotonicity theorem

Theorem 4.5.

$$\Gamma \vdash A, \Gamma \subseteq \Delta \implies \Delta \vdash A$$

Proof. Let the proof of A from Γ be the sequence $\langle \phi_1, \phi_2, \dots, \phi_n \rangle, \phi_n = A$

Base case: $n = 1$

- Case 1: $\phi_1 \in \Gamma \implies \phi_1 \in \Delta \implies \Delta \vdash \phi_1$
- Case 2: ϕ_1 is from one of the axiom schema, $\Delta \vdash \phi_1$

Inductive step: $\forall i < n, \Gamma \vdash \phi_i, \Delta \vdash \phi_i$

- Case 1, 2: $\phi_{i+1} \in \Gamma$ or ϕ_{i+1} is from one of the axiom schema, trivial
- Case 3: ϕ_{i+1} is from Modus Ponens, $\exists \phi_j$ such that

$$\frac{\phi_j \rightarrow \phi_{i+1} \quad \phi_j}{\phi_{i+1}}$$

$\Gamma \vdash \phi_j, \Gamma \vdash \phi_j \rightarrow \phi_{i+1}$ by inductive hypothesis, $\Delta \vdash \phi_j, \Delta \vdash \phi_j \rightarrow \phi_{i+1}$, by MP, $\Delta \vdash \phi_{i+1}$

□

4.2.2 Deduction

Theorem 4.6.

$$\Gamma \cup \{A\} \vdash B \iff \Gamma \vdash A \rightarrow B$$

Proof. ($\implies$)

We will use structural induction over the length of derivation from $\Gamma \cup \{A\}$ to B , $\Gamma \cup \{A\} \vdash B \implies \exists \langle \phi_1, \phi_2, \dots, \phi_n \rangle, \phi_n = B$.

Base case: $n = 1, \phi_n = B$

- Case 1: B is an axiom $\Gamma \vdash B$. By simplification axiom, $\Gamma \vdash B \rightarrow (A \rightarrow B)$, by MP, $\Gamma \vdash A \rightarrow B$.
- Case 2: $B = A$, we will use the simplification and distribution axioms:

- $p \equiv A$
- $q \equiv (A \rightarrow A)$
- $r \equiv A$
- $(A \rightarrow ((A \rightarrow A) \rightarrow A)) \rightarrow ((A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A))$ (Distribution)
- $A \rightarrow ((A \rightarrow A) \rightarrow A)$ (Simplification)
- $(A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A)$ (MP)
- $A \rightarrow (A \rightarrow A)$ (Simplification)
- $(A \rightarrow A)$ (MP)

$$\Gamma \vdash A \rightarrow A \implies \Gamma \vdash A \rightarrow B$$

Inductive step: Suppose $\forall i < n, \Gamma \cup \{A\} \vdash \phi_i \implies \Gamma \vdash A \rightarrow \phi_i$

- Case 1, 2, 3: ϕ_{i+1} is in Γ or ϕ_{i+1} is from one of the axioms, trivial; If $\phi_{i+1} = A$, the same as Case 2 in base case
- Case 4: ϕ_{i+1} is obtained from MP, where $\exists j \leq i$,

$$\begin{array}{ll} \Gamma \cup \{A\} \vdash \phi_j & \implies \Gamma \vdash A \rightarrow \phi_j \\ \Gamma \cup \{A\} \vdash \phi_j \rightarrow \phi_{i+1} & \implies \Gamma \vdash A \rightarrow (\phi_j \rightarrow \phi_{i+1}) \end{array}$$

$$\frac{\phi_j \rightarrow \phi_{i+1} \quad \phi_j}{\phi_{i+1}}$$

$$\begin{array}{ll} \Gamma \vdash (A \rightarrow \phi_j) \rightarrow (A \rightarrow \phi_{i+1}) & \text{by distribution} \\ \Gamma \vdash A \rightarrow \phi_{i+1} & \text{by MP} \end{array}$$

□

Theorem 4.7 (Soundness of HSPS). *HSPS is sound.*

$$\Gamma \vdash \phi \implies \Gamma \models \phi$$

Proof. Define property $H(i) : \forall j \leq i, \Gamma \models c_j$.

Base case: $i = 1$

Suppose $\Gamma \vdash \phi, \exists \pi = c_1, c_2, \dots, c_k, c_k = \phi$. By definition, $c_1 \in \Gamma$ or

- $\exists \phi, \psi, c_1 \equiv \phi \rightarrow (\psi \rightarrow \phi)$
- $\exists \phi, \psi, \rho, c_1 \equiv [\phi \rightarrow (\psi \rightarrow \rho)] \rightarrow [(\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \rho)]$
- $\exists \phi, c_1 \equiv ((\phi \rightarrow \perp) \rightarrow \perp) \rightarrow \phi$

Note that Modus Ponens is not possible to yield c_1 as we need at least 1 previous formula.

If $c_1 \in \Gamma, \Gamma \models c_1$; Else, if c_1 comes from any other axiom, c_1 is valid, i.e. $\forall v, v \models c_1$, therefore $\Gamma \models c_1$

Inductive step:

Suppose $H(i), \forall j < i, \Gamma \models c_j, \forall v \models \Gamma, v \models c_j$.

- Case 1, 2: $c_{i+1} \in \Gamma$, or c_{i+1} is from an axiom, $\Gamma \models c_{i+1}, H(i+1)$
- Case 3: c_{i+1} comes from MP: $\exists j < i, \Gamma \vdash c_j, \Gamma \vdash c_j \rightarrow c_{i+1}$. By inductive hypothesis, $\forall v \models \Gamma, v \models c_j, v \models c_j \rightarrow c_{i+1}$

$$\begin{array}{l} v(c_j) = T \\ v(c_j \rightarrow c_{i+1}) = T \equiv v(\neg c_j \vee c_{i+1}) = T \\ v(\neg c_j) = T \text{ OR } v(c_{i+1}) = T \\ v(c_j) = T \implies v(\neg c_j) = F \implies v(c_{i+1}) = T \\ \implies \forall v \models \Gamma, v \models c_{i+1} \implies \Gamma \models c_{i+1} \end{array}$$

□

Theorem 4.8 (Completeness of HSPS). *HSPS is complete.*

$$\Gamma \models \phi \implies \Gamma \vdash \phi$$

4.3 Weak Completeness and Completeness of HSPS

We work with the Hilbert-Style Propositional System (HSPS):

- Axiom 1: $p \rightarrow (q \rightarrow p)$
- Axiom 2: $(p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))$
- Axiom 3: $((p \rightarrow \perp) \rightarrow \perp) \rightarrow p$
- Rule: Modus Ponens

We write $\vdash \varphi$ for provability and $\models \varphi$ for semantic validity.

1. Weak Completeness

Theorem 4.9 (Weak Completeness). *If $\models \varphi$, then $\vdash \varphi$.*

Idea. We prove the contrapositive: if $\not\vdash \varphi$, then $\not\models \varphi$. That is, if φ is not provable, then there exists a valuation falsifying it.

Step 1: Consistency and Maximal Consistency. A set Γ is *consistent* if $\Gamma \not\vdash \perp$.

If $\not\vdash \varphi$, then $\{\neg\varphi\}$ is consistent.

Using a standard Lindenbaum construction:

- Extend any consistent set Γ
- to a *maximal consistent set* Δ
- such that for every formula ψ :

$$\psi \in \Delta \quad \text{or} \quad \neg\psi \in \Delta.$$

This construction relies only on classical logic and the deduction theorem (derivable in HSPS).

Step 2: Canonical Valuation. Define a valuation v_Δ by:

$$v_\Delta(p) = \begin{cases} \text{true} & \text{if } p \in \Delta \\ \text{false} & \text{otherwise} \end{cases}$$

We prove by structural induction:

$$\psi \in \Delta \quad \Longleftrightarrow \quad v_\Delta \models \psi.$$

The key steps use:

- Maximality of Δ
- Closure under Modus Ponens
- Axiom 3 for classical negation

Step 3: Falsifying φ . Since $\neg\varphi \in \Delta$, we obtain:

$$v_\Delta \models \neg\varphi, \quad \text{hence} \quad v_\Delta \not\models \varphi.$$

Therefore φ is not valid.

Conclusion.

$$\models \varphi \implies \vdash \varphi.$$

□

2. Strong Completeness (Full Completeness)

Theorem 4.10 (Completeness). *If $\Gamma \models \varphi$, then $\Gamma \vdash \varphi$.*

Reduction to Weak Completeness. Suppose $\Gamma \models \varphi$.

Then:

$$\models \bigwedge \Gamma \rightarrow \varphi.$$

If Γ is finite, we directly apply weak completeness:

$$\vdash (\bigwedge \Gamma) \rightarrow \varphi.$$

Repeated applications of Modus Ponens yield:

$$\Gamma \vdash \varphi.$$

Infinite Case. If Γ is infinite:

- If $\Gamma \models \varphi$,
- then some finite subset $\Gamma_0 \subseteq \Gamma$ satisfies $\Gamma_0 \models \varphi$ (compactness of propositional logic).

Apply the finite case to Γ_0 , then use monotonicity of derivability:

$$\Gamma \vdash \varphi.$$

3. Conceptual Structure

Weak completeness:

$$\models \varphi \Rightarrow \vdash \varphi.$$

Strong completeness:

$$\Gamma \models \varphi \Rightarrow \Gamma \vdash \varphi.$$

The essential engine is:

1. Maximal consistent sets
2. Canonical valuation
3. Truth lemma

Axiom 3 ensures classicality, making the canonical model work.

4. Final Picture

Soundness:

$$\vdash \varphi \Rightarrow \models \varphi.$$

Completeness:

$$\models \varphi \Rightarrow \vdash \varphi.$$

Hence:

$$\vdash \varphi \iff \models \varphi.$$

HSPS is therefore sound and complete for classical propositional logic.

■

4.4 Resolution Proof System

Definition 4.11 (Clause). For each proposition p define literals p and $\neg p$. A clause C is finite set of literals over the proposition set P .

$$v \models C = \{\ell_1, \ell_2, \dots, \ell_k\} \iff \exists j, v \models \ell_j$$

As such, a clause C is satisfiable if $\exists v, v \models C$

Note that from this definition, an empty clause (an empty set of literals) is not satisfiable, as intuitively, a valuation models a clause if there exists at least one literal in the clause such that $v \models \ell$.

Definition 4.12 (Conjunctive normal form (CNF)). A formula ϕ is said to be in Conjunctive Normal Form is

$$\phi \equiv \bigwedge_{i=1}^k C_i$$

Definition 4.13 (Satisfiability of a set of clauses).

$$\Gamma = \{C_1, C_2, \dots\}$$

- Γ is satisfiable if $\exists v, \forall C \in \Gamma, v \models C$
- Γ is unsatisfiable if $\forall v, \exists C \in \Gamma, v \not\models C$

By this definition,

- $\Gamma = \emptyset$ is vacuously satisfiable
- $\Gamma = \{\emptyset\}$ is not satisfiable.
- Any set of clauses that contains the empty set is unsatisfiable

Theorem 4.14. Each well-formed formula can be transformed to a logically equivalent CNF

Proof. **Base case:** $\phi \in P$

trivial

Inductive Case 1: $\neg$

Suppose there exists a CNF $\bigwedge_i \bigvee_j \ell_{i,j}$ logically equivalent to ϕ .

$$\begin{aligned} \neg \phi &= \neg((\ell_{1,1} \vee \ell_{1,2} \vee \dots) \wedge (\ell_{2,1} \vee \ell_{2,2} \vee \dots) \wedge \dots) \\ &= \neg(\ell_{1,1} \vee \ell_{1,2} \vee \dots) \vee \neg(\ell_{2,1} \vee \ell_{2,2} \vee \dots) \vee \dots \\ &= (\neg \ell_{1,1} \wedge \neg \ell_{1,2} \wedge \dots) \vee (\neg \ell_{2,1} \wedge \neg \ell_{2,2} \wedge \dots) \vee \dots \\ &= (\neg \ell_{1,1} \vee \neg \ell_{2,1} \vee \dots) \wedge () \end{aligned}$$

Inductive Case 2: $\wedge$

Suppose $\phi \equiv \bigwedge_{i \in I} C_i$ and $\psi \equiv \bigwedge_{j \in J} D_j$ are CNFs.

$$\begin{aligned} \phi \wedge \psi &\equiv \left(\bigwedge_{i \in I} C_i \right) \wedge \left(\bigwedge_{j \in J} D_j \right) \\ &\equiv \bigwedge_{k \in I \cup J} E_k \end{aligned}$$

where E_k is a clause from the set of clauses of ϕ or ψ . Since the conjunction of two conjunctions is simply a larger conjunction of the original clauses, the result is by definition a CNF.

Inductive Case 3: $\vee$

Suppose $\phi \equiv \bigwedge_{i \in I} C_i$ and $\psi \equiv \bigwedge_{j \in J} D_j$ are CNFs. By applying the Distributive Law:

$$\begin{aligned}\phi \vee \psi &\equiv \left(\bigwedge_{i \in I} C_i \right) \vee \left(\bigwedge_{j \in J} D_j \right) \\ &\equiv \bigwedge_{i \in I} \left(C_i \vee \left(\bigwedge_{j \in J} D_j \right) \right) \\ &\equiv \bigwedge_{i \in I} \left(\bigwedge_{j \in J} (C_i \vee D_j) \right) \\ &\equiv \bigwedge_{i \in I, j \in J} (C_i \vee D_j)\end{aligned}$$

Since each C_i and D_j are disjunctions of literals, their union $C_i \vee D_j$ is also a disjunction of literals, and thus a valid clause. The resulting expression is a conjunction of these clauses, satisfying the definition of a CNF.

Conclusion

By the principle of induction on the complexity of the formula, we have shown that if the sub-formulas of ϕ have equivalent CNFs, then $\neg\phi$, $\phi \wedge \psi$, and $\phi \vee \psi$ also have equivalent CNFs. Since the set of connectives $\{\neg, \wedge, \vee\}$ is functionally complete, every well-formed formula can be transformed into a logically equivalent CNF. We can bring all negations to literal level, and supply an additional negation to each literal to negate the whole formula. $\square$

The Resolution Rule

The system relies on a single rule of inference. Given two clauses C' and D' such that $C' = C \oplus \{p\}$ and $D' = D \oplus \{\neg p\}$ (where $\oplus$ denotes a disjoint union), the **resolvent** with respect to p is defined as the clause:

$$Res(C', D') = C \cup D$$

Resolution Proofs

A resolution proof of Γ is a finite sequence of clauses $\pi = C_1, C_2, \dots, C_n$ where the final clause C_n is the empty clause $\square$. For every clause C_i in the sequence, one of the following must be true:

1. C_i is an original clause from the set Γ ($C_i \in \Gamma$).
2. C_i is the resolvent of two previous clauses C_j and C_k (where $j, k < i$) with respect to some proposition p .

Soundness and Completeness

Theorem 4.15 (Soundness). *The resolution proof system is sound:*

$$\Gamma \vdash_{RES} \square \implies \Gamma \text{ is unsatisfiable}$$

Proof. Suppose there exists a resolution derivation $C_1, C_2, \dots, C_k$ where $C_k = \square$. We proceed by induction on the length of the derivation k .

Base Case: $k = 1$

If the empty clause $\square$ is in the derivation at step 1, then $\square \in \Gamma$. Since an empty clause is by definition unsatisfiable (it is an empty disjunction), Γ is unsatisfiable.

Inductive Step

Assume any clause derived in $i \leq k$ steps is a semantic consequence of Γ . Consider C_{k+1} derived from C_a and C_b ($a, b \leq k$) using the resolution rule on literal p :

$$C_a = \{p\} \cup C'_a, \quad C_b = \{\neg p\} \cup C'_b, \quad C_{k+1} = C'_a \cup C'_b$$

By the Inductive Hypothesis, $\Gamma \models C_a$ and $\Gamma \models C_b$. Let v be a valuation satisfying Γ .

- If $v(p) = F$, then since $v(C_a) = T$, it must be that $v(C'_a) = T$.
- If $v(p) = T$, then since $v(C_b) = T$, it must be that $v(C'_b) = T$.

In either case, $v(C'_a \cup C'_b) = T$. Thus $\Gamma \models C_{k+1}$. If $C_{k+1} = \square$, then $\Gamma \models \perp$, meaning Γ has no model and is unsatisfiable. $\square$

Theorem 4.16 (Completeness). *The resolution proof system is complete:*

$$\Gamma \text{ is unsatisfiable} \implies \Gamma \vdash_{RES} \square$$

Proof. We prove this by induction on the number of distinct propositional variables n in Γ .

Base Case: $n = 0$

If Γ is unsatisfiable and has 0 variables, it must contain the empty clause $\square$. Thus $\Gamma \vdash_{RES} \square$.

Inductive Step

Pick a variable p and define two sets:

- $\Gamma_{p=T}$: The set of clauses obtained by setting p to True (remove clauses containing p , remove $\neg p$ from remaining).
- $\Gamma_{p=F}$: The set of clauses obtained by setting p to False (remove clauses containing $\neg p$, remove p from remaining).

If Γ is unsatisfiable, then both $\Gamma_{p=T}$ and $\Gamma_{p=F}$ are unsatisfiable. By the Inductive Hypothesis, there exist resolution proofs for $\square$ from both sets. By re-inserting p and $\neg p$, these proofs yield $\Gamma \vdash_{RES} \{p\}$ and $\Gamma \vdash_{RES} \{\neg p\}$. Resolving these two results yields $\square$. $\square$

5 Complexity

5.1 Turing Machine

Definition 5.1 (Turing Machine).

$$\mathcal{M} = \langle Q, \Sigma, \Gamma, \delta, q_0, q_{Acc}, q_{Rej}, k \rangle$$

- Q : the set of states
- Σ : finite input alphabet
- Γ : finite worktap alphabet, $\square \in \Gamma$
- δ : the set of transitions
- q_0 : starting state, $q_0 \in Q$
- q_{Acc} : accepting state, $q_{Acc} \in Q$
- q_{Rej} : rejecting state, $q_{Rej} \in Q$ (Optional)
- k : the number of worktaps (Optional)

A TM has 1 read-only input tape, may have 1 write only output tape, and k work tape. We can show that any finitely many k tapes is equivalent to 1 worktap TM. [TODO]

Transifiton function

$$\delta = \langle (Q \times \Sigma \times \Gamma^k) \times (Q \times \{L, R, S\} \times \Gamma^k \times \{L, R, S\}^k \times \Gamma \times \{L, R, S\}) \rangle$$

A state transition

$$\tau = \langle q, \sigma, \gamma_1, \gamma_2, \dots, \gamma_k, q', d, \gamma'_1, \gamma'_2, \dots, \gamma'_k, d_1, d_2, \dots, d_k, \gamma', d_o \rangle$$

$$\begin{aligned} \tau = & \langle q, \sigma, \\ & \gamma_1, \gamma_2, \dots, \gamma_k, \\ & q', d, \\ & \gamma'_1, \gamma'_2, \dots, \gamma'_k, \\ & d_1, d_2, \dots, d_k, \\ & \gamma', d_o \rangle \end{aligned}$$

Represents a transition from state q , when the head reading input symbol σ , and worktap symbol γ_i on worktape i , transition into state q' , write worktape symbol γ'_i , move the input head in the direction d , and workhead i in direction d_i , writing γ' on the output tape and move output head in direction d_o

Non-determiniestic TM

Non deterministic TM has non-deterministic transition funcitons

$$\delta \subseteq \langle (Q \times \Sigma \times \Gamma^k) \times (Q \times \{L, R, S\} \times \Gamma^k \times \{L, R, S\}^k \times \Gamma \times \{L, R, S\}) \rangle$$

5.2 Semantics of TM

Definition 5.2 (Configuration). A configuration C_t of a Turing machine at step t captures the overall state of the TM. Now we only consider TM with no rejecting states and output tape.

$$\begin{aligned} C = & \langle q, \\ & w_{in}^1 \nabla w_{in}^2, \\ & w_{w1}^1 \nabla w_{w1}^2 \\ & w_{w2}^1 \nabla w_{w2}^2 \\ & \dots \\ & w_{wk}^1 \nabla w_{wk}^2 \rangle \end{aligned}$$

Where $w^1 \nabla w^2 = w$, the symbol read by the head at each configuration is the first symbol after ∇ . Note that ∇ is not a Symbol but just an indication of the head position.

The initial configuration will starts at the starting state q_0 , input head and work heads all at the beginning, and all worktapes empty.

$$C_0 = \langle q_0, \nabla w_{in}, \underbrace{\nabla, \nabla, \dots, \nabla}_k \rangle$$

Configuration

$$\begin{aligned} C_{t+1} = & \langle q, \\ & w_{in}^1 \nabla w_{in}^2, \\ & w_{w1}^1 \nabla w_{w1}^2 \\ & w_{w2}^1 \nabla w_{w2}^2 \\ & \dots \\ & w_{wk}^1 \nabla w_{wk}^2 \rangle \end{aligned}$$

is a valid successor of

$$C_t = \langle q', \\ w_{in}^{1'} \nabla w_{in}^{2'}, \\ w_{w1}^{1'} \nabla w_{w1}^{2'} \\ w_{w2}^{1'} \nabla w_{w2}^{2'} \\ \dots \\ w_{wk}^{1'} \nabla w_{wk}^{2'} \rangle$$

$$\exists \tau = \langle p, \sigma, \gamma_1, \gamma_2, \dots, \gamma_k, p', d, \gamma'_1, \gamma'_2, \dots, \gamma'_k, d_1, d_2, \dots, d_k, \gamma', d_o \rangle$$

such that:

- σ is the first symbol after the input head: $w_{in}^2[0]$
- γ_i is the first symbol after work head i : $w_{wi}^2[0]$
- - if $d = S$: $w'_{in} = w_{in}$
 $|w_{in}^1| - |w_{in}^{1'}| = 0$
 - if $d = L$: $w'_{in} = w_{in}^1[: -1] \nabla w_{in}^1[-1] \cdot w_{in}^2$
 $|w_{in}^1| - |w_{in}^{1'}| = 1$
 - if $d = L$: $w'_{in} = w_{in}^1 \cdot w_{in}^2[0] \nabla w_{in}^2[1 :]$
 $|w_{in}^1| - |w_{in}^{1'}| = -1$
- $\forall i \in [1, k]$,
 - if $d_i = S$: $w'_{wi} = w_{wi}$
 $|w_{wi}^1| - |w_{wi}^{1'}| = 0$
 - if $d_i = L$: $w'_{wi} = w_{wi}^1[: -1] \nabla w_{wi}^1[-1] \cdot w_{wi}^2$
 $|w_{wi}^1| - |w_{wi}^{1'}| = 1$
 - if $d_i = L$: $w'_{wi} = w_{wi}^1 \cdot w_{wi}^2[0] \nabla w_{wi}^2[1 :]$
 $|w_{wi}^1| - |w_{wi}^{1'}| = -1$

Definition 5.3. $\forall w \in \Sigma^*$, $\mathcal{M}$ yields some valid sequences of configurations, which is called a run $\pi_w = C_0, C_1, \dots$ where

- C_0 is the initial configuration
- $\forall t \geq 1, C_t$ is a valid successor of C_{t-1}

A run is accepting if

1. π is finite, and
2. π ends with a configuration C_n , where $C_n.q = q_{Acc}$

A run is rejecting if

- π is infinite, or
- π ends with a configuration C_n , where $C_n.q \neq q_{Acc}$

Definition 5.4. $w \in \Sigma^*$ is accepted by $\mathcal{M}$ if $\exists \pi_w, \pi_w$ is accepting.

Now we can bridge to the concept of languages,

Definition 5.5. The set of strings accepted by $\mathcal{M}$ is the language described by $\mathcal{M}$, ie

$$L(\mathcal{M}) = \{w \in \Sigma^* \mid w \text{ is accepted by } \mathcal{M}\}$$

Now we will focus on decidable languages, and their recognizing non deterministic TM.

Definition 5.6 (Terminating Turing Machine). $\mathcal{M}$ is *terminating* if $\forall w \in \Sigma^*, \forall \pi_w, \pi_w$ is finite.

5.3 Running time and Space

Definition 5.7.

$$T : \mathbb{N} \mapsto \mathbb{N}$$

The running time of $\mathcal{M}$ is bounded by T if

$$\forall w \in \Sigma^*, \forall \pi_w, |\pi_w| \leq T(|w|)$$

Definition 5.8 (Space usage). The space usage of at configuration C_i is

$$space(C_i) = |w_{out}| + \sum_{j=1}^k |w_{wj}|$$

Definition 5.9.

$$S : \mathbb{N} \mapsto \mathbb{N}$$

The space usage of $\mathcal{M}$ is bounded by S if

$$\forall w \in \Sigma^*, \forall \pi_w, \forall C \in \pi_w, space(C) \leq S(|w|)$$

Definition 5.10. L is *computable* by $\mathcal{M}$ if $L = L(\mathcal{M})$

Definition 5.11. L is *computable in time T* if $\exists \mathcal{M}$

1. $L = L(\mathcal{M})$ and
2. the running time of $\mathcal{M}$ is bounded by T

Definition 5.12. L is *computable in space S* if $\exists \mathcal{M}$

1. $L = L(\mathcal{M})$ and
2. the space usage of $\mathcal{M}$ is bounded by S

Theorem 5.13 (Church-Turing Thesis). Any model of computation is only as powerful as the Turing machine.
 $\forall L, \exists N$ in a new model of computation, $L(N) = L \implies \exists \mathcal{M} \in TM, L(\mathcal{M}) = L$

5.4 Complexity classes

Definition 5.14 (NTIME).

$$NTIME(T(\cdot)) = \{L \mid L \text{ is computable in time } T(\cdot)\}$$

Definition 5.15 (DTIME).

$$DTIME(T(\cdot)) = \{L \mid \exists \text{ deterministic } \mathcal{M}, L(\mathcal{M}) = L, \mathcal{M} \text{ running time is bounded by } T(\cdot)\}$$

Definition 5.16 (NSPACE).

$$NSPACE(S(\cdot)) = \{L \mid L \text{ is computable in space } S(\cdot)\}$$

Definition 5.17 (*DSPACE*).

$$DSPACE(S(\cdot)) = \{L \mid \exists \text{ deterministic } \mathcal{M}, L(\mathcal{M}) = L, \mathcal{M} \text{ space usage is bounded by } S(\cdot)\}$$

$$\begin{aligned} P &= PTIME = \bigcup_{k \geq 0} DTIME(f_k), & f_k(n) &= n^k \\ PSPACE &= \bigcup_{k \geq 0} DSPACE(f_k), & f_k(n) &= n^k \\ NP &= NPTIME = \bigcup_{k \geq 0} NTIME(f_k), & f_k(n) &= n^k \\ NPSPACE &= \bigcup_{k \geq 0} NSPACE(f_k), & f_k(n) &= n^k \end{aligned}$$

Theorem 5.18.

$$PTIME \subseteq NPTIME$$

Theorem 5.19.

$$PSPACE \subseteq NPSPACE$$

Theorem 5.20.

$$NPSPACE \subseteq PSPACE$$

Proof.

□

Corollary 5.21.

$$PSPACE = NPSPACE$$

5.5 Hardness

Definition 5.22 (Hardness of Turing machine). $\mathcal{M}_1 \leq \mathcal{M}_2$ if all runs of $\mathcal{M}_1$ are smaller than $\mathcal{M}_2$

5.6 Reduction

Theorem 5.23 (Constant speed up theorem). L is computable by $f \implies L$ is computable by f' , where

$$f'(n) = f\left(\frac{n}{c}\right)$$

Theorem 5.24. For all non-deterministic turing machines $\mathcal{M}_1$ with, there exists a deterministic turing machine $\mathcal{M}_2$, such that $L(\mathcal{M}_1) = L(\mathcal{M}_2)$

Lemma 5.25. For all deterministic turing machine with $\mathcal{M}_1$ with k worktapes, there exists a deterministic turing machine $\mathcal{M}_2$ with 1 worktape such that $L(\mathcal{M}_1) = L(\mathcal{M}_2)$. Same for non deterministic TM.

Consider deterministic TM $\mathcal{M}$ with an output tape, it induces function

$$f_{\mathcal{M}} : \Sigma^* \mapsto \Gamma^*$$

Such that

$$f_{\mathcal{M}} = \begin{cases} \text{undefined, if } \mathcal{M} \text{ does not terminate;} \\ u, \text{ if } \mathcal{M} \text{ terminates, such that in the final configuration, output tape contains } u \end{cases}$$

Corollary 5.26. $f_{\mathcal{M}}$ is total if $\mathcal{M}$ is terminating

Theorem 5.27. A partial function $f : \Sigma^* \mapsto \Gamma^*$ is computable is $\exists \mathcal{M}$ that induces $f_{\mathcal{M}}, f = f_{\mathcal{M}}$

Theorem 5.28. A total function is computable in time $T : \mathbb{N} \mapsto \mathbb{N}$ if there exists $\mathcal{M}$ such that $f_{\mathcal{M}} = f$, and $\mathcal{M}$ is running time bounded by T

5.7 Cook–Levin Theorem

Theorem 5.29 (Cook–Levin). *SAT is NP-complete. In particular, for every nondeterministic Turing machine $\mathcal{M}$ running in polynomial time, there exists a polynomial-time reduction mapping any input w to a Boolean formula $\Phi_{\mathcal{M},w}$ such that*

$$w \in L(\mathcal{M}) \iff \Phi_{\mathcal{M},w} \text{ is satisfiable.}$$

Proof Outline.

1. $\text{SAT} \in \text{NP}$.
2. Every language in NP reduces to SAT .

We focus on (2) under the Turing machine semantics defined above.

Step 1: $\text{SAT} \in \text{NP}$

Given a Boolean formula φ , guess a valuation and evaluate φ in time polynomial in $|\varphi|$. Hence $\text{SAT} \in \text{NP}$.

Step 2: Reduction from NP to SAT

Let $L \in \text{NP}$. Then there exists a nondeterministic Turing machine $\mathcal{M}$ and a polynomial $p(n)$ such that for every input w :

$$w \in L \iff \mathcal{M} \text{ has an accepting run on } w \text{ of length } \leq p(|w|).$$

Fix w with $|w| = n$, and let $T = p(n)$. We construct a Boolean formula $\Phi_{\mathcal{M},w}$ expressing:

“There exists an accepting run of $\mathcal{M}$ on w of length $\leq T$.”

Encoding Configurations

Since $\mathcal{M}$ runs for at most T steps:

- The input head moves at most T cells.
- Each work tape head moves at most T cells.

Thus each tape region used is bounded by $O(T)$ cells.

We represent the entire computation as a *tableau*:

$$\text{Time } 0, 1, \dots, T \quad \times \quad \text{Tape positions } -T, \dots, T$$

Each row corresponds to a configuration.

Propositional Variables

We introduce variables:

- $Q_{t,q}$: machine is in state q at time t .
- $H_{t,x}^i$: head of tape i is at position x at time t .
- $S_{t,x,\gamma}^i$: tape i at position x contains symbol γ at time t .

Here $i = 0$ denotes the input tape and $i = 1, \dots, k$ the work tapes.

The number of variables is polynomial in T .

Constraints

The formula $\Phi_{\mathcal{M},w}$ is a conjunction of four parts.

(A) Exactly-One Constraints For every time t :

- Exactly one state holds:

$$\bigvee_{q \in Q} Q_{t,q} \quad \wedge \quad \bigwedge_{q \neq q'} (\neg Q_{t,q} \vee \neg Q_{t,q'})$$

- Exactly one head position per tape.
- Exactly one symbol per tape cell.

(B) Initial Configuration We enforce:

- Q_{0,q_0} .
- Input tape contains w .
- Work tapes contain blanks.
- All heads at starting position.

All of these are expressed as unit clauses.

(C) Transition Constraints For every $t < T$ and every possible local configuration, we encode the transition relation.

Fix a transition

$$\tau = \langle q, \sigma, \gamma_1, \dots, \gamma_k, q', d, \gamma'_1, \dots, \gamma'_k, d_1, \dots, d_k \rangle.$$

For each position x , we enforce:

If at time t :

- state is q ,
- input head at x reading σ ,
- work head i at x_i reading γ_i ,

then at time $t + 1$:

- state is q' ,
- written symbols updated,
- heads moved according to d, d_i ,
- all other cells unchanged.

Each such condition is encoded by clauses of the form:

$$(\text{current configuration}) \rightarrow (\text{next configuration})$$

which is translated into CNF using Tseitin transformation.

(D) Acceptance We require that for some $t \leq T$:

$$Q_{t,q_{Acc}}$$

This is encoded as:

$$\bigvee_{t=0}^T Q_{t,q_{Acc}}.$$

Correctness

Soundness If $\mathcal{M}$ has an accepting run of length $\leq T$, assign variables according to that run.

All constraints (A)–(D) are satisfied. Hence $\Phi_{\mathcal{M},w}$ is satisfiable.

Completeness If $\Phi_{\mathcal{M},w}$ is satisfiable, the satisfying assignment defines:

- A unique state per time,
- A unique head position per time,
- A unique symbol per cell,

and transition constraints ensure each configuration is a valid successor of the previous one. Thus we extract a valid accepting run of $\mathcal{M}$ on w . Hence $w \in L(\mathcal{M})$.

Size and Complexity

- Time bound $T = p(n)$.
- Tableau size $O(T^2)$.
- Number of variables and clauses polynomial in T .

Therefore $\Phi_{\mathcal{M},w}$ can be constructed in polynomial time.

Conclusion

We have shown:

$$L \in \text{NP} \implies L \leq_p \text{SAT}.$$

Since $\text{SAT} \in \text{NP}$, we conclude:

SAT is NP-complete.

□

6 Syntax of First Order Logics

Let consider the limitation of propositional logics. Propositional logics intuitively represents choices, and each proposition has no internal structure, it is simply a boolean value to be evaluated. And since formula needs to be finitely long, we cannot express problems of infinite size. Thus we introduce first order logic, which has constant, variables and quantifiers, allowing us to express more nuanced ideas.

6.1 Vocabulary of FOL

1. Signature: $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R})$
 - Constant symbols: $\mathcal{C}$
 - Function symbols: $\mathcal{F}$
 - Relation symbols: $\mathcal{R}$
2. Connector: $C = \{\neg, \vee\}$
3. Variables: V
4. Quantifiers: $\{\forall, \exists\}$
5. Equality: $=$ as a special relation

Definition 6.1. *Term Similar to how we define formula in propositional logic, a term can also be recursively defined as*

$$t := c \in \mathcal{C} \mid x \in V \mid f(\underbrace{t, \dots}_{\text{arity}(f)}), f \in \mathcal{F}$$

Here we use term to represent the idea of an "object" in FOL.

Definition 6.2. *Formula*

$$\phi := t = t \mid \underbrace{R(t, \dots)}_{\text{arity}(R)}, R \in \mathcal{R} \mid \phi \vee \phi \mid \neg \phi \mid \exists x \in V, \phi \mid \forall x \in V, \phi$$

6.2 Modelling with FOL

Example 6.3. *Number line*

$$\Sigma = (\{0\}, \{succ\}, \{<\})$$

Under this signature, we can write formula:

- $\phi_1 = \forall x \exists y, y = succ(x)$
- $x = 0$
- $0 = 1$

These are all syntactically valid formula.

7 Semantics of First Order Logics

In propositional logics, semantics are conveyed by valuations, which assigns true or false to each proposition. In FOL, we need to define the universe and interpret the symbols to unpack the semantics.

Definition 7.1. *Structure*

$$\mathfrak{A} = (U, I)$$

A structure is a tuple of U , an non-empty universe set, and I , the interpretation of symbols.

Definition 7.2. *Interpretation* An interpretation is a mapper from symbols to elements in the universe.

- For every constant $c \in \mathcal{C}$, $I(c) \in U$
- For every function symbol $f \in \mathcal{F}$, $I(f) \in [U^k \mapsto U]$, $k = \text{arity}(f)$
- For every relation symbol $R \in \mathcal{R}$, $I(R) \subseteq U^k$, $k = \text{arity}(R)$

Definition 7.3. *Assignment* Given a structure $\mathfrak{A} = (U, I)$

$$\alpha \in [V \mapsto U]$$

Is an assignment of each variable to elements in the universe.

Definition 7.4. *Valuation of terms*

$$\begin{aligned} Val_{\mathfrak{A}, \alpha}(c) &= I(c) \in U \\ Val_{\mathfrak{A}, \alpha}(x) &= \alpha(x) \in U \\ Val_{\mathfrak{A}, \alpha}(f(t, \dots)) &= I(f)(Val_{\mathfrak{A}, \alpha}(t), \dots) \in U \end{aligned}$$

Essentially Valuation is an assignment of elements in a universe to terms, the objects in FOL

Definition 7.5. *Substitution*

$$\alpha[x \mapsto u](z) = \begin{cases} u, & \text{if } z = x \\ \alpha(z), & \text{otherwise} \end{cases}$$

Definition 7.6. *Valuation of Formula*

Base case: $\phi \equiv t_1 = t_2$

$$\mathfrak{A}, \alpha \models \phi \iff \text{Val}_{\mathfrak{A}, \alpha}(t_1) = \text{Val}_{\mathfrak{A}, \alpha}(t_2)$$

Inductive cases:

$$1. \phi \equiv R(\underbrace{t, \dots}_{\text{arity}(R)})$$

$$\mathfrak{A}, \alpha \models \phi \iff \underbrace{(\text{Val}_{\mathfrak{A}, \alpha}(t), \dots)}_{\text{arity}(R)} \in I(R)$$

$$2. \phi \equiv \neg \varphi$$

$$\mathfrak{A}, \alpha \models \phi \iff \mathfrak{A}, \alpha \not\models \varphi$$

$$3. \phi \equiv \phi_1 \vee \phi_2$$

$$\mathfrak{A}, \alpha \models \phi \iff \mathfrak{A}, \alpha \models \phi_1 \text{ OR } \mathfrak{A}, \alpha \models \phi_2$$

$$4. \phi \equiv \exists x, \varphi$$

$$\mathfrak{A}, \alpha \models \phi \iff \text{there is } u \in U, \mathfrak{A}, \alpha[x \mapsto u] \models \varphi$$

$$5. \phi \equiv \forall x, \varphi$$

$$\mathfrak{A}, \alpha \models \phi \iff \text{for all } u \in U, \mathfrak{A}, \alpha[x \mapsto u] \models \varphi$$

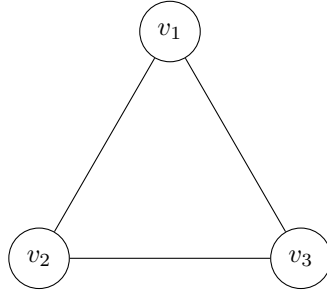
Example 7.7. graphs Consider signature

$$\Sigma = (\{s, t, u\}, \emptyset, \{E\})$$

We can come up with a structure

$$\begin{aligned} \mathfrak{A} &= (U_G, I_G) \\ U_G &= \{v_1, v_2, v_3\} \\ I_G(s) &= v_1 \\ I_G(t) &= v_2 \\ I_G(u) &= v_3 \\ I_G(E) &= \{(v_1, v_2), (v_2, v_3), (v_3, v_1)\} \end{aligned}$$

The graph represented by $\mathfrak{A}$ is:



Definition 7.8. Variables and Free variables Variables in a term is defined as:

$$\text{Var}(x) = x$$

$$\text{Var}(c) = \emptyset$$

$$\text{Var}(f(t_1, \dots, t_k)) = \bigcup_{i=1}^k \text{Var}(t_i)$$

Free variables in a formula can be defined as:

$$\text{FVar}(\forall x, \phi) = \text{FVar}(\phi) \setminus \{x\}$$

$$\text{FVar}(\exists x, \phi) = \text{FVar}(\phi) \setminus \{x\}$$

In words, free variables are variables appear in a formula that are not bounded by quantifiers

Definition 7.9. *Satisfaction of FOL formula*

$$\phi \text{ is satisfiable} \iff \exists(\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \phi$$

Definition 7.10. *Logical Consequence*

$$\Gamma \models \phi \iff \forall(\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \Gamma \implies \mathfrak{A}, \alpha \models \phi$$

Lemma 7.11 (Relevance Lemma for FOL). *Let ϕ be a FOL formula, let $\mathfrak{A}$ be a structure and let α_1, α_2 be variable assignments such that for all $x \in FVar(\phi)$,*

$$\alpha_1(x) = \alpha_2(x).$$

Then

$$\mathfrak{A}, \alpha_1 \models \phi \iff \mathfrak{A}, \alpha_2 \models \phi.$$

Proof. By symmetry, we only need to prove if for all $x \in FVar(\phi)$, $\alpha_1(x) = \alpha_2(x)$ then

$$\mathfrak{A}, \alpha_1 \models \phi \implies \mathfrak{A}, \alpha_2 \models \phi.$$

Lemma 7.12 (Substitution lemma).

$$\mathfrak{A}, \alpha \models \phi[x \mapsto t] \iff \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] \models \phi$$

Lemma 7.13 (Substitution lemma).

$$\forall x \in Var(t), \alpha_1(x) = \alpha_2(x) \implies Val_{\mathfrak{A}, \alpha_1}(t) = Val_{\mathfrak{A}, \alpha_2}(t)$$

Lemma 7.14 (Substitution lemma).

$$\mathfrak{A} \models \psi[x \mapsto t] \iff \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}}(t)] \models \psi$$

Lemma 7.15.

$$Val_{\mathfrak{A}, \alpha}(t[x \mapsto t']) = Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')]}(t)$$

Proof. **Base case:**

1. $t = c \in C$:

$$Val_{\mathfrak{A}, \alpha}(t[x \mapsto t']) = Val_{\mathfrak{A}, \alpha}(c) = I(c)$$

$$Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')]}(t) = Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')]}(c) = I(c)$$

2. $t = y \in V$:

Case 1: $x = y$

$$\implies y[x \mapsto t'] = t'$$

$$Val_{\mathfrak{A}, \alpha}(y[x \mapsto t']) = Val_{\mathfrak{A}, \alpha}(t')$$

$$Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')]}(y) = \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')](y) = Val_{\mathfrak{A}, \alpha}(t')$$

Case 2: $x \neq y$

$$\implies y[x \mapsto t'] = y$$

$$Val_{\mathfrak{A}, \alpha}(y[x \mapsto t']) = Val_{\mathfrak{A}, \alpha}(y) = \alpha(y)$$

$$Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')]}(y) = \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')](y) = \alpha(y)$$

Inductive step: $t = f(t_1, \dots)$

$$\begin{aligned} t[x \mapsto t'] &= f(t_1[x \mapsto t'], t_2[x \mapsto t'], \dots) \\ Val_{\mathfrak{A}, \alpha}(t[x \mapsto t']) &= Val_{\mathfrak{A}, \alpha}(f(t_1[x \mapsto t'], t_2[x \mapsto t'], \dots)) = \\ &= I(f)(Val_{\mathfrak{A}, \alpha}(t_1[x \mapsto t']), Val_{\mathfrak{A}, \alpha}(t_2[x \mapsto t']), \dots) \end{aligned}$$

The other side

$$Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t')]}(f(t_1, t_2, \dots)) = I(f)(Val_{\mathfrak{A}, \alpha}(t_1[x \mapsto t']), Val_{\mathfrak{A}, \alpha}(t_2[x \mapsto t']), \dots)$$

□

Proof. We prove the substitution lemma for formulae by structural induction on ψ .

Base case: $\psi \equiv t_1 = t_2$

$$\psi[x \mapsto t] \equiv t_1[x \mapsto t] = t_2[x \mapsto t]$$

Thus

$$\mathfrak{A}, \alpha \models \psi[x \mapsto t] \iff Val_{\mathfrak{A}, \alpha}(t_1[x \mapsto t]) = Val_{\mathfrak{A}, \alpha}(t_2[x \mapsto t]).$$

By the substitution lemma for terms,

$$Val_{\mathfrak{A}, \alpha}(t_1[x \mapsto t]) = Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)]}(t_1)$$

and

$$Val_{\mathfrak{A}, \alpha}(t_2[x \mapsto t]) = Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)]}(t_2).$$

Hence

$$Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)]}(t_1) = Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)]}(t_2),$$

that is,

$$\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] \models t_1 = t_2.$$

Inductive steps

1. $\psi \equiv R(t_1, \dots, t_n)$.

$$\psi[x \mapsto t] \equiv R(t_1[x \mapsto t], \dots, t_n[x \mapsto t]).$$

Then

$$\mathfrak{A}, \alpha \models \psi[x \mapsto t] \iff (Val_{\mathfrak{A}, \alpha}(t_1[x \mapsto t]), \dots, Val_{\mathfrak{A}, \alpha}(t_n[x \mapsto t])) \in I(R).$$

By the substitution lemma for terms, for each i ,

$$\begin{aligned} Val_{\mathfrak{A}, \alpha}(t_i[x \mapsto t]) &= Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)]}(t_i). \\ (Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)]}(t_1), \dots, Val_{\mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)]}(t_n)) &\in I(R), \\ \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] &\models R(t_1, \dots, t_n). \end{aligned}$$

2. $\psi \equiv \neg\phi$.

$$\begin{aligned} \mathfrak{A}, \alpha \models (\neg\phi)[x \mapsto t] &\iff \mathfrak{A}, \alpha \models \neg(\phi[x \mapsto t]) \\ &\iff \mathfrak{A}, \alpha \not\models \phi[x \mapsto t]. \end{aligned}$$

By IH,

$$\begin{aligned} \mathfrak{A}, \alpha \models \phi[x \mapsto t] &\iff \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] \models \phi. \\ \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] &\not\models \phi, \\ \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] &\models \neg\phi. \end{aligned}$$

3. $\psi \equiv \phi_1 \vee \phi_2$.

$$\begin{aligned} \mathfrak{A}, \alpha \models (\phi_1 \vee \phi_2)[x \mapsto t] &\iff \mathfrak{A}, \alpha \models \phi_1[x \mapsto t] \vee \phi_2[x \mapsto t]. \\ \mathfrak{A}, \alpha \models \phi_1[x \mapsto t] \quad \text{or} \quad \mathfrak{A}, \alpha \models \phi_2[x \mapsto t]. \end{aligned}$$

By IH

$$\begin{aligned} \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] \models \phi_1 \quad \text{or} \quad \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] \models \phi_2. \\ \mathfrak{A}, \alpha[x \mapsto Val_{\mathfrak{A}, \alpha}(t)] \models \phi_1 \vee \phi_2. \end{aligned}$$

□

□

Lemma 7.16 (Analogous relevance lemma on structure). *If $\mathfrak{A}$ and $\mathfrak{B}$ agree on all parameters occur in ϕ , then $\mathfrak{A} \models \phi \iff \mathfrak{B} \models \phi$*
If

$$\begin{aligned} \forall c \in C, Val_{\mathfrak{A}, \alpha}(c) &= Val_{\mathfrak{B}, \alpha}(c) \\ \forall x \in V, Val_{\mathfrak{A}, \alpha}(x) &= Val_{\mathfrak{B}, \alpha}(x) \\ \forall f(t, \dots), Val_{\mathfrak{A}, \alpha}(f(t, \dots)) &= Val_{\mathfrak{B}, \alpha}(f(t, \dots)) \\ \forall R(t, \dots), R(t, \dots) \in I_{\mathfrak{A}}(R) &\iff R(t, \dots) \in I_{\mathfrak{B}}(R) \\ \mathfrak{A} \models \phi &\iff \mathfrak{B} \models \phi \end{aligned}$$

8 Compactness of First Order Logics

Definition 8.1. *Compactness Theorem for First Order Logics*

$$\begin{aligned} \Gamma \text{ is satisfiable} &\iff \Gamma \text{ is finitely satisfiable} \\ \Gamma \text{ is satisfiable} &\iff \forall \Gamma' \subseteq_{fin} \Gamma, \exists (\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \Gamma' \end{aligned}$$

Theorem 8.2. *Structures are not countable, and the uncountability is uncapped, as U and I can be arbitrarily defined*

Definition 8.3. *Prenex Normal Form ϕ is in Prenex Normal Form if all quantifiers are in the beginning of the formula.*

$$\phi \equiv Q_1 x_1 Q_2 x_2 \dots \psi$$

Where ψ is the matrix of ϕ , which is a quantifier-free formula.

8.1 Prenex Normal Form

1. **Remove implication and biconditional.**

Replace

$$\begin{aligned} A \rightarrow B &\equiv \neg A \vee B \\ A \leftrightarrow B &\equiv (A \rightarrow B) \wedge (B \rightarrow A) \end{aligned}$$

2. **Push negations inward (Negation Normal Form).**

Apply De Morgan's laws and quantifier negation:

$$\begin{aligned} \neg(A \wedge B) &\equiv \neg A \vee \neg B \\ \neg(A \vee B) &\equiv \neg A \wedge \neg B \\ \neg \forall x \varphi &\equiv \exists x \neg \varphi \\ \neg \exists x \varphi &\equiv \forall x \neg \varphi \end{aligned}$$

After this step, negations appear only directly in front of atomic formulas.

3. Standardize variables apart: α -renaming

Rename bound variables so that each quantifier binds a unique variable and no variable is bound by more than one quantifier.

4. Move quantifiers outward.

Move quantifiers across logical connectives when the variable does not occur free in the other formula. For example,

$$\begin{aligned} (\forall x A) \wedge B &\equiv \forall x(A \wedge B) \quad \text{if } x \notin \text{FV}(B) \\ (\exists x A) \vee B &\equiv \exists x(A \vee B) \quad \text{if } x \notin \text{FV}(B) \end{aligned}$$

Continue moving quantifiers outward until all quantifiers appear at the front of the formula.

5. Obtain Prenex Normal Form.

The formula now has the form

$$Q_1 x_1 Q_2 x_2 \dots Q_n x_n \varphi$$

where each $Q_i \in \{\forall, \exists\}$ and φ is quantifier-free.

Example 8.4.

$$\phi \equiv P(x) \wedge \forall z(R(x, z) \vee \exists y Q(y))$$

1. move out quantifiers:

$$\begin{aligned} \phi &\equiv P(x) \wedge \forall z \exists y (R(x, z) \vee Q(y)) \\ \phi &\equiv \forall z \exists y [P(x) \wedge (R(x, z) \vee Q(y))] \end{aligned}$$

2. PNF obtained, $\varphi \equiv P(x) \wedge (R(x, z) \vee Q(y))$

Example 8.5.

$$\phi \equiv \forall x(\neg P(x) \vee \exists y(Q(x, y) \wedge \forall z R(y, z))) \vee \exists x S(x)$$

1. α -renaming

$$\phi \equiv \forall x(\neg P(x) \vee \exists y(Q(x, y) \wedge \forall z R(y, z))) \vee \exists u S(u)$$

2.

Lemma 8.6. ψ , the PNF of ϕ is logically equivalent to ϕ

$$\forall(\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \psi \iff \mathfrak{A}, \alpha \models \phi$$

Proof.

□

Definition 8.7. Universally quantified formula ϕ is universally quantified if all quantifiers in ϕ are $\forall$. Analogous to Existential quantified formula

Definition 8.8. Sentence ϕ is a sentence if there is no free variables.

$$\text{FVar}(\phi) = \emptyset$$

To determine satisfiability, we can ignore existential quantifiers.

Theorem 8.9. Let φ be a formula and $\text{FVar}(\varphi) = \{x_1, x_2, \dots, x_k\}$ φ is satisfiable iff

$$\varphi' \equiv \exists x_1 \exists x_2, \dots, \exists x_k \varphi \text{ is satisfiable}$$

φ' is a sentence

Definition 8.10. Skolemization Transform PNF into universally quantified PNF.

$\forall \phi, \exists \phi'$, such that

1. ϕ and ϕ' are equisatisfiable. (Not necessarily equivalent!)

8.2 Skolemization

1. Convert the formula to prenex normal form.

Transform the formula so that all quantifiers appear at the front:

$$Q_1 x_1 Q_2 x_2 \dots Q_n x_n \varphi$$

where φ is quantifier-free.

2. Process quantifiers from left to right.

For each existential quantifier $\exists x$, determine the universal variables that precede it in the prefix.

3. Replace existential variables with Skolem terms.

- If $\exists x$ has no preceding universal quantifiers, replace x with a new Skolem constant c .

$$\exists x P(x) \Rightarrow P(c)$$

- If $\exists x$ is preceded by universal variables $u_1, \dots, u_k$, replace x with a new Skolem function $f(u_1, \dots, u_k)$.

$$\forall u_1 \dots \forall u_k \exists x P(x, u_1, \dots, u_k) \Rightarrow P(f(u_1, \dots, u_k), u_1, \dots, u_k)$$

4. Remove the existential quantifier.

After substitution with the Skolem term, delete the corresponding existential quantifier from the prefix.

5. Repeat until no existential quantifiers remain.

The resulting formula contains only universal quantifiers.

6. Optionally drop universal quantifiers.

In many applications (e.g., resolution), the remaining universal quantifiers are omitted, since variables are implicitly assumed to be universally quantified.

Example 8.11.

$$\phi \equiv \exists x P(x) \wedge \forall z (R(x, z) \vee \exists y Q(y))$$

1. convert to PNF

$$\phi \equiv \exists x \forall z \exists y (P(x) \wedge (R(x, z) \vee Q(y)))$$

2. if the leftmost quantifier is existential, replace the variable with a new constant c

$$\phi' \equiv \forall z \exists y (P(c_x) \wedge (R(c_x, z) \vee Q(y)))$$

$$c_x \notin \Sigma$$

At this stage, ϕ and ϕ' are equisatisfiable. No longer equivalent

Proof. ϕ is satisfiable $\implies \exists \mathfrak{A}, \alpha \models \phi$. We can construct a new signature and structure from it as such

$$\Sigma' = \Sigma \cup \{c_x\}, \quad I'(c_x) = \alpha(x)$$

We can show that $\mathfrak{A}', \alpha \models \phi'$

The other way can be done similarly □

3. if the next existential quantifier is not the outer most, add a new function symbol to replace each variable quantified by the existential quantifier with the function call using all previous universally quantified variables

$$\phi'' \equiv \forall z (P(c_x)) \wedge (R(c_x, z) \vee Q(f_y(z)))$$

We can show that ϕ' and ϕ'' are equisatisfiable

Theorem 8.12. *Skolemization preserves satisfiability.*

Proof. Consider skolemization as a recursive algorithm described as follows:

Algorithm 1: Skolemization on Prenex Formula

Input: Signature $\Sigma = (C, F, R)$;
Formula $\phi = (\pi, \psi)$ where π is a quantifier prefix and ψ is quantifier-free;
List u of universally quantified variables in scope (initially empty).
Output: Skolemized formula (π', ψ') equisatisfiable with ϕ .
if π *is empty* **then**
 return $(\emptyset, \psi)$
 $(q, x) \leftarrow \text{head}(\pi)$;
 $\pi_{\text{rest}} \leftarrow \text{tail}(\pi)$
if $q = \exists$ **then**
 if u *is empty* **then**
 introduce fresh constant c_x ;
 $\Sigma.C \leftarrow \Sigma.C \uplus \{c_x\}$;
 return $\text{Skol}(\Sigma, (\pi_{\text{rest}}, \psi[x \mapsto c_x]), u)$
 else
 introduce fresh function f_x of arity $|u|$;
 $\Sigma.F \leftarrow \Sigma.F \uplus \{f_x\}$;
 let $t = f_x(u_1, \dots, u_k)$ where $u = [u_1, \dots, u_k]$;
 return $\text{Skol}(\Sigma, (\pi_{\text{rest}}, \psi[x \mapsto t]), u)$
else
 // $q = \forall$
 $u' \leftarrow u \cdot x$ // **append** x **to** u
 $(\pi', \psi') \leftarrow \text{Skol}(\Sigma, (\pi_{\text{rest}}, \psi), u')$;
 return $((\forall x) \cdot \pi', \psi')$

We claim that the invariant is the satisfiability of input and output is the same. Let $\text{Skol}(\Sigma, (\pi, \psi), u)$ be the recursive algorithm defined. We prove by induction on the length of the quantifier prefix $n = |\pi|$ that the algorithm preserves equisatisfiability.

Inductive Hypothesis (IH)

For any prefix π of length n , any signature Σ , and any list of universal variables $u = [u_1, \dots, u_k]$, a formula $\phi = \forall u(\pi\psi)$ is satisfiable in a Σ -structure $\mathfrak{A}$ if and only if there exists a larger $\mathfrak{A}'$ than $\mathfrak{A}$ to Σ' (containing new symbols) such that $\mathfrak{A}'$ satisfies the output of $\text{Skol}(\Sigma, (\pi, \psi), u)$.

$$\mathfrak{A} \models \phi = \forall u(\pi\psi) \iff \mathfrak{A}' \models \text{Skol}(\Sigma, (\pi, \psi), u)$$

Case 1: Base Case ($n = 0$)

If π is empty, the algorithm returns $(\emptyset, \psi)$, trivially equisatisfiable

Case 2: Universal Case ($q = \forall$)

Let $\pi = \forall x \cdot \pi_{\text{rest}}$. The algorithm sets $u' = u \cdot x$ and recurses

We will show that:

$$\mathfrak{A} \models \forall u \forall x (\pi_{\text{rest}} \psi) \iff \mathfrak{A}' \models \forall u \forall x (\text{Skol}(\pi_{\text{rest}}, \psi, u'))$$

- ($\Rightarrow$): Suppose $\mathfrak{A} \models \forall u \forall x (\pi_{\text{rest}} \psi) \implies \mathfrak{A} \models \forall (u \cup \{x\}) (\pi_{\text{rest}} \psi)$. There exists an expansion $\mathfrak{A}'$ such that $\mathfrak{A}' \models \forall (u \cup \{x\}) \text{Skol}(\pi_{\text{rest}}, \psi, u')$. rearrange the quantifiers gives $\mathfrak{A}' \models \forall u \forall x \text{Skol}(\dots)$.
- ($\Leftarrow$): Suppose $\mathfrak{A}' \models \text{Skol}(\pi_{\text{rest}}, \psi, u')$. Since $\mathfrak{A}'$ is an expansion of $\mathfrak{A}$, and the universal quantifier $\forall x$ does not introduce new symbols in this step, same logic

Case 3: Existential Case ($q = \exists$)

□

Theorem 8.13 (Herbrand's Theorem). *Informally, if a first-order formula ϕ is satisfiable, then its satisfiability is witnessed by a special kind of structure called a Herbrand structure.*

Definition 8.14 (Herbrand structure). *Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{P})$ be a signature. A structure*

$$\mathfrak{A} = (U, I)$$

is a Herbrand structure if

1. *The universe is the Herbrand universe*

$$U = T_{\mathcal{C} \cup \mathcal{F}},$$

the set of all ground terms constructed from the constant symbols $\mathcal{C}$ and function symbols $\mathcal{F}$.

2. *The interpretation I is fixed syntactically:*

- *For every constant symbol $c \in \mathcal{C}$,*

$$I(c) = c$$

- *For every function symbol $f \in \mathcal{F}$ of arity n ,*

$$I(f)(t_1, \dots, t_n) = f(t_1, \dots, t_n)$$

Definition 8.15. *Ground term*

$$gt := c \mid f(gt, \dots)$$

Basically terms without variables

Definition 8.16. *Ground Atom (Let's ignore = first)*

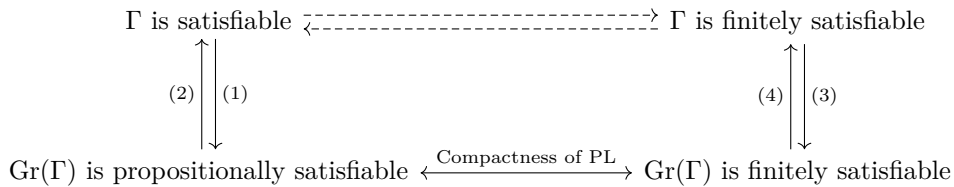
$$ga := R(gt, \dots)$$

Definition 8.17 (Ground Formula).

$$gf := gt = gt \mid R(gt, \dots) \mid \neg gf \mid gf \vee gf$$

No variables, no quantifiers

Now that we have developed the machinery needed to prove Compactness for first-order logic, we outline the overall proof strategy. From the definition of grounding, it is clear that the goal is to connect first-order logic with propositional logic. By grounding, we eliminate variables and quantifiers, and treat each ground atom as a propositional atom. In this way, we translate the satisfiability problem for a first-order theory into a corresponding satisfiability problem in propositional logic. This suggests the following proof plan: establish a correspondence between satisfiability and finite satisfiability for Γ and for its grounding $\text{Gr}(\Gamma)$, then apply Compactness of propositional logic to $\text{Gr}(\Gamma)$ in order to deduce Compactness for first-order logic.



Definition 8.18 (Ground instantiation). *ϕ is a universally quantified sentence in PNF, let ψ be a ground formula, with the same signature. ψ is a ground instantiation of ϕ , or $\text{Gr}(\phi) = \psi$ if:*

$$\phi = \forall x_1 \forall x_2, \dots, \forall x_n \cdot \gamma$$

There are n ground terms $gt_1, gt_2, \dots, gt_n$, such that

$$\psi = \gamma[x_1 \mapsto gt_1][x_2 \mapsto gt_2] \dots [x_n \mapsto gt_n]$$

Definition 8.19 (Ground instantiation of a set).

$$Gr(\Gamma) = \bigcup_{\phi \in \Gamma} Gr(\phi)$$

Lemma 8.20 (Ground instantiation of a formula is countable).

Corollary 8.21 (Ground instantiation of a set is countable).

Definition 8.22 (Ground valuation).

$$gv : ga_\Sigma \mapsto \{true, false\}$$

$$gv \models_{gr} R(t, \dots) \iff gv(R(t, \dots)) = true$$

$$gv \models_{gr} \neg gf \iff gv \not\models_{gr} gf$$

$$gv \models_{gr} gf_1 \vee gf_2 \iff gv \models_{gr} gf_1 \text{ OR } gv \models_{gr} gf_2$$

Definition 8.23 (Propositionally satisfiable). A ground formula gf is propositionally satisfiable if

$$\exists \tau \models_{gr} gf$$

Definition 8.24 (Propositionally satisfiable set). A set of ground formula Γ is propositionally satisfiable if there exist a ground atomic valuation τ such that

$$\forall \phi \in \Gamma, \tau \models_{Gr} \phi$$

8.3 Proof of the compactness theorem for FOL

Let's first consider the case without equality, the definition of ground formula becomes:

$$gf := R(ga, \dots) \mid \neg gf \mid gf \vee gf$$

8.3.1 Γ is satisfiable $\implies Gr(\Gamma)$ is propositionally satisfiable

Lemma 8.25 (Substitution Lemma for Ground Terms). Let $\mathfrak{A} = (U, I)$ be a structure, ψ a formula, x a variable, and t a ground term. Then for any assignment α ,

$$\mathfrak{A}, \alpha \models \psi[x \mapsto t] \iff \mathfrak{A}, \alpha[x \mapsto \text{Val}_{\mathfrak{A}}(t)] \models \psi.$$

Proof. We have proven a more general case of substitution for FOL formula and terms □

Lemma 8.26 (Ground Instantiation for Universal Prefix). Let $\mathfrak{A} = (U, I)$ be a structure and

$$\phi = \forall x_1 \dots \forall x_n \psi.$$

If $\mathfrak{A} \models \phi$, then for all ground terms $t_1, \dots, t_n$,

$$\mathfrak{A} \models \psi[x_1 \mapsto t_1] \dots [x_n \mapsto t_n].$$

Proof. Assume $\mathfrak{A} \models \forall x_1 \dots \forall x_n \psi$.

Let $t_1, \dots, t_n$ be arbitrary ground terms. Define

$$u_i = \text{Val}_{\mathfrak{A}}(t_i) \in U \quad (1 \leq i \leq n).$$

By the semantics of universal quantifiers,

$$\mathfrak{A} \models \forall x_1 \cdots \forall x_n \psi \implies \mathfrak{A}, \alpha[x_1 \mapsto u_1] \cdots [x_n \mapsto u_n] \models \psi$$

for every assignment α .

By the substitution lemma for ground terms,

$$\mathfrak{A}, \alpha \models \psi[x_1 \mapsto t_1] \cdots [x_n \mapsto t_n] \iff \mathfrak{A}, \alpha[x_1 \mapsto u_1] \cdots [x_n \mapsto u_n] \models \psi.$$

Hence,

$$\mathfrak{A}, \alpha \models \psi[x_1 \mapsto t_1] \cdots [x_n \mapsto t_n].$$

Since α was arbitrary,

$$\mathfrak{A} \models \psi[x_1 \mapsto t_1] \cdots [x_n \mapsto t_n].$$

□

Lemma 8.27 (Ground Semantic Equivalence). *Let $\mathfrak{A} = (U, I)$ and τ be defined as above. Then for every ground formula χ ,*

$$\mathfrak{A} \models \chi \iff \tau \models_{Gr} \chi.$$

Proof. Trivial by definition

□

Lemma 8.28 (Ground Semantic Equivalence). *Let $\mathfrak{A} = (U, I)$ and τ be defined as above. Then for every ground formula χ ,*

$$\mathfrak{A} \models \chi \iff \tau \models_{Gr} \chi.$$

Proof. We prove the claim by structural induction on the ground formula χ .

Base case: χ is a ground atom. Then

$$\mathfrak{A} \models \chi \iff \tau \models_{Gr} \chi$$

by the Atomic Agreement Lemma.

Inductive case 1: $\chi = \neg\chi_1$ for some ground formula χ_1 .

By the semantics of negation,

$$\mathfrak{A} \models \neg\chi_1 \iff \mathfrak{A} \not\models \chi_1.$$

By the induction hypothesis,

$$\mathfrak{A} \not\models \chi_1 \iff \tau \not\models_{Gr} \chi_1.$$

Again by the semantics of negation,

$$\tau \not\models_{Gr} \chi_1 \iff \tau \models_{Gr} \neg\chi_1.$$

Hence

$$\mathfrak{A} \models \neg\chi_1 \iff \tau \models_{Gr} \neg\chi_1.$$

Inductive case 2: $\chi = \chi_1 \vee \chi_2$ for some ground formulas χ_1, χ_2 .

By the semantics of disjunction,

$$\mathfrak{A} \models \chi_1 \vee \chi_2 \iff (\mathfrak{A} \models \chi_1 \text{ or } \mathfrak{A} \models \chi_2).$$

By the induction hypothesis,

$$(\mathfrak{A} \models \chi_1 \text{ or } \mathfrak{A} \models \chi_2) \iff (\tau \models_{Gr} \chi_1 \text{ or } \tau \models_{Gr} \chi_2).$$

Again by the semantics of disjunction,

$$(\tau \models_{Gr} \chi_1 \text{ or } \tau \models_{Gr} \chi_2) \iff \tau \models_{Gr} \chi_1 \vee \chi_2.$$

Hence

$$\mathfrak{A} \models \chi_1 \vee \chi_2 \iff \tau \models_{Gr} \chi_1 \vee \chi_2.$$

Therefore, for every ground formula χ ,

$$\mathfrak{A} \models \chi \iff \tau \models_{Gr} \chi.$$

□

Lemma 8.29 (Grounding Preserves Truth). *Let Γ be a set of sentences (in Skolem form). If*

$$\mathfrak{A} \models \Gamma,$$

then

$$\mathfrak{A} \models \text{Ground}(\Gamma).$$

Lemma 8.30 (Transfer to Ground Valuation). *Let $\mathfrak{A} = (U, I)$ and τ be defined as above. If*

$$\mathfrak{A} \models \text{Ground}(\Gamma),$$

then

$$\tau \models_{Gr} \text{Ground}(\Gamma).$$

Proof. Γ is satisfiable $\implies$ there exists a structure $\mathfrak{A}$ such that

$$\mathfrak{A} \models \Gamma$$

Define ground valuation on atoms

$$\tau : ga_{\Sigma} \mapsto \{true, false\}$$

and for all ground atoms over Σ , define

$$\tau(a) = true \iff \mathfrak{A} \models a$$

Now we will show that: $\mathfrak{A} \models \phi \implies \tau \models_{Gr} \phi$

Define ground valuation τ as such:

$$\tau(R(t_1, \dots, t_n)) = \begin{cases} true & \text{if } (Val_{\mathfrak{A}}(t_1), \dots, Val_{\mathfrak{A}}(t_n)) \in I_{\mathfrak{A}}(R) \\ false & \text{otherwise} \end{cases}$$

Base case: ϕ is ground atom

By definition, trivial

Inductive steps: induction over the structure of formula(not gf)

- $\varphi = \neg\phi$: $\mathfrak{A} \models \neg\phi \implies \mathfrak{A} \not\models \phi$.
- $\varphi = \phi_1 \vee \phi_2$

□

8.4 The Resolution Proof System for First-Order Logic

The resolution proof system is a refutation-based logic used for automated theorem proving. It generalizes propositional resolution to First-Order Logic (FOL) by incorporating unification. Instead of proving that a formula ϕ is valid, we show that $\neg\phi$ is unsatisfiable.

8.4.1 The Resolution Rule and Unification

In FOL, resolution operates on clauses in Skolem Normal Form (SNF). Given two clauses with no variables in common:

$$C_1 = \{L_1, \dots, L_k\} \quad \text{and} \quad C_2 = \{M_1, \dots, M_n\}$$

If there exist literals $L_i \in C_1$ and $M_j \in C_2$ such that L_i and $\neg M_j$ are unifiable with a Most General Unifier (MGU) σ , the **resolvent** is:

$$\text{Res}(C_1, C_2) = (C_1 \setminus \{L_i\})\sigma \cup (C_2 \setminus \{M_j\})\sigma$$

8.4.2 Soundness of FOL Resolution

Theorem 8.31 (Soundness). *If there exists a resolution derivation of the empty clause $\square$ from a set of clauses $\mathcal{C}$, then $\mathcal{C}$ is unsatisfiable.*

Proof Sketch. The proof is based on the fact that the resolution rule is truth-preserving under any interpretation:

1. Let $\mathfrak{A}$ be a model of $\mathcal{C}$.
2. If $\mathfrak{A} \models C_1$ and $\mathfrak{A} \models C_2$, then for any substitution σ , $\mathfrak{A} \models C_1\sigma$ and $\mathfrak{A} \models C_2\sigma$.
3. Since $L_i\sigma$ and $M_j\sigma$ are complementary literals, any assignment that satisfies both clauses must satisfy the remaining literals in the resolvent.
4. By induction, if $\square$ (the empty clause, which is always false) is derived, no model $\mathfrak{A}$ can exist.

□

8.4.3 Completeness of FOL Resolution

Theorem 8.32 (Refutational Completeness). *If a set of clauses $\mathcal{C}$ is unsatisfiable, then there exists a resolution derivation of the empty clause $\square$.*

Proof Sketch. The proof typically utilizes Herbrand's Theorem and the Lifting Lemma:

1. **Herbrand's Theorem:** If $\mathcal{C}$ is unsatisfiable, there exists a finite set of ground instances $\mathcal{C}_{ground}$ that is unsatisfiable in propositional logic.
2. **Propositional Completeness:** By the completeness of propositional resolution, there is a derivation of $\square$ from $\mathcal{C}_{ground}$.
3. **Lifting Lemma:** For every resolution step between ground instances C'_1, C'_2 at the propositional level, there exists a corresponding resolution step between the general clauses C_1, C_2 at the FOL level using unification.
4. Therefore, the propositional derivation of $\square$ can be "lifted" to a first-order derivation of $\square$.

□

8.5 Special FOL fragments

8.5.1 Quantifier-free FOL

Definition 8.33 (quantifier free formula).

$$\phi := t = t \mid R(t, \dots) \mid \neg\phi \mid \phi \vee \phi$$

All variables are free

Definition 8.34 (QT-free SAT). $QT\text{-}F\text{ SAT} = \{\phi \mid \phi \text{ is quantifier free, } \phi \text{ is satisfiable}\}$

Definition 8.35 (Small Model Property). *A fragment $\mathbb{F}$ of logic is said to have the **Small Model Property (SMP)** if there exists a computable function $b_{\mathbb{F}} : \mathbb{F} \rightarrow \mathbb{N}$ such that for every formula $\phi \in \mathbb{F}$:*

$$\text{If } \phi \text{ is satisfiable, then } \exists \mathfrak{A} \models \phi \text{ such that } |U_{\mathfrak{A}}| \leq b_{\mathbb{F}}(\phi)$$

We are interested in QT-free formula because it satisfy the SMP

Theorem 8.36. *If $\mathbb{F}$ satisfy SM, the satisfiability of $\mathbb{F}$ is decidable.*

Corollary 8.37. $FORM_{FOL}$ does not have SMP

counter example. Consider the following formula:

$$\phi \equiv \forall x, S(x) \neq 0 \wedge (x \neq y \rightarrow S(x) \neq S(y))$$

This basically describes a structure that has no right endpoint, and no 2 unique elements have the same successor. This requires an infinite (unbounded) structure, thus $FORM_{FOL}$ does not possess the SMP $\square$

Proof. Notice that, for quantifier free formula, the set of ground instantiations is finite.

$$\phi \in QT - freeSAT \iff skol(\phi) \in QT - freeSAT \wedge \phi \text{ is grounded}$$

As skolemization preserves satisfiability $\square$

8.6 Homomorphism and Isomorphism

Definition 8.38 (Homomorphism). Let $\Sigma = (C, F, R)$ be a First order signature. $\mathfrak{A}_1 = (U_1, I_1)$ and $\mathfrak{A}_2 = (U_2, I_2)$ be Σ -structures. A function

$$h : U_1 \mapsto U_2$$

is a homomorphism from $\mathfrak{A}_1$ to $\mathfrak{A}_2$ if:

1. $\forall c \in C, h(I_1(c)) = I_2(c)$
2. $\forall f^k \in F, \forall (u_1, \dots, u_k) \in U_1^k, h(I_1(f)(u_1, \dots, u_k)) = I_2(f)(h(u_1), \dots, h(u_k))$
3. $\forall R^k \in R, \forall (u_1, \dots, u_k) \in U_1^k, (u_1, \dots, u_k) \in I_1(R) \iff (h(u_1), \dots, h(u_k)) \in I_2(R)$

Definition 8.39 (Isomorphism). Let $\Sigma = (C, F, R)$ be a First order signature. $\mathfrak{A}_1 = (U_1, I_1)$ and $\mathfrak{A}_2 = (U_2, I_2)$ be Σ -structures. A function

$$h : U_1 \mapsto U_2$$

is an isomorphism from $\mathfrak{A}_1$ to $\mathfrak{A}_2$ if:

1. h is a homomorphism from $\mathfrak{A}_1$ to $\mathfrak{A}_2$
2. h is a bijection

denoted as

$$\mathfrak{A}_1 \cong \mathfrak{A}_2$$

Theorem 8.40 (Isomorphism Theorem).

$$\mathfrak{A}_1, \alpha \models \phi \iff \mathfrak{A}_2, (h \circ \alpha) \models \phi$$

8.7 Theory

Definition 8.41 (Theory). A theory T is a set of sentences.

Example 8.42 (Partial order).

$$T_{po} = \begin{cases} \forall x \forall y \forall z < (x, y) \wedge < (y, z) \rightarrow < (x, z) & (\text{transitivity}) \\ \forall x \neg < (x, x) & (\text{irreflexivity}) \end{cases}$$

Definition 8.43 (T-satisfiable). Let T be a theory over Σ , a formula ϕ over Σ is T -satisfiable if

$$\exists \mathfrak{A}, \alpha, \mathfrak{A}, \alpha \models \phi$$

AND

$$\forall \psi \in T, \mathfrak{A} \models \psi$$

Definition 8.44 (T-validity). *Let T be a theory over Σ , a formula ϕ over Σ is T -valid if*

$$\forall \mathfrak{A}, \text{ if } \forall \psi \in T, \mathfrak{A} \models \psi \rightarrow \mathfrak{A} \models \phi$$

8.7.1 Common clauses used to define theories

- $\sigma_1 \equiv \forall x \neg(x < x) \wedge \forall x \forall y \forall z (x < y \wedge y < z \rightarrow x < z)$ (partial order)
- $\sigma_2 \equiv \forall x \forall y (x < y \vee x = y \vee y < x)$ (trichotomy)
- $\sigma_3 \equiv \forall x \forall y (x < y \rightarrow \exists z (x < z < y))$ (dense)
- $\sigma_4 \equiv \forall x \exists y \exists z (y < x < z)$ (no endpoints)
- $\varphi_1 \equiv \forall x \forall y \forall z ((x + y) + z) = x + (y + z)$ (associativity)
- $\varphi_2 \equiv \forall x ((x + 0 = x = 0 + x) \wedge (x + (-x) = (-x) + x = 0))$ (additive identity & add. inverse axioms)
- $\varphi_3 \equiv \forall x \forall y (x + y = y + x)$ (commutativity)
- $\varphi_4 \equiv \forall x \forall y \forall z (x < y \rightarrow ((x + z) < (y + z)))$ (order-compatibility axiom for addition)
- $\psi_n \equiv \forall x (nx = 0 \rightarrow x = 0)$ (torsion-freeness)

Theories formed over above sentences include:

- $\mathfrak{A} \models \sigma_1 \iff \mathfrak{A}$ is a partial order
- $\mathfrak{A} \models \{\sigma_1, \sigma_2\} \iff \mathfrak{A}$ is a linear/total order
- $\mathfrak{A} \models \{\sigma_1, \sigma_2, \sigma_3, \sigma_4\} \iff \mathfrak{A}$ is a dense linear order without endpoints
- $\mathfrak{A} \models \{\varphi_1, \varphi_2\} \iff \mathfrak{A}$ is a group
- $\mathfrak{A} \models \{\varphi_1, \varphi_2, \varphi_3\} \iff \mathfrak{A}$ is an abelian group
- $\mathfrak{A} \models \{\varphi_1, \varphi_2, \varphi_3, \varphi_4, \sigma_1, \sigma_2\} \iff \mathfrak{A}$ is an ordered abelian group
- $\mathfrak{A} \models \{\varphi_1, \varphi_2, \varphi_3, \psi_n\}_{n \in \mathbb{N}} \iff \mathfrak{A}$ is a torsion-free abelian group

8.7.2 Monotonicity under theory inclusion

Lemma 8.45. *If $T_1 \subseteq T_2$, ϕ is T_1 -valid $\implies \phi$ is T_2 -valid. Or*

$$T_1 \subseteq T_2, T_1 \models \phi \implies T_2 \models \phi$$

Lemma 8.46. *If $T_1 \subseteq T_2$, ϕ is T_2 -sat $\implies \phi$ is T_1 -sat. Or*

$$T_1 \subseteq T_2, T_2 \cup \{\phi\} \text{ is satisfiable } \implies T_1 \cup \{\phi\} \text{ is satisfiable}$$

$$T_1 \subseteq T_2, T_1 \cup \{\phi\} \text{ is unsatisfiable } \implies T_2 \cup \{\phi\} \text{ is unsatisfiable}$$

8.7.3 Theory of a structure

Instead of a set of sentences, we can also define a theory with a structure.

Definition 8.47. *Let $\mathfrak{A}$ be a structure,*

$$Th(\mathfrak{A}) = \{\phi \mid \phi \text{ is a sentence, } \mathfrak{A} \models \phi\}$$

Definition 8.48 (Class of structures). *Let $\mathfrak{C}$ be a collection of structure,*

$$Th(\mathfrak{C}) = \bigcap_{\mathfrak{A} \in \mathfrak{C}} Th(\mathfrak{A})$$

Definition 8.49.

$$Th(\Gamma) = \{\phi \mid \phi \text{ is a sentence}, \Gamma \models \phi\}$$

Theorem 8.50.

$$Th(Th(\Gamma)) \subseteq Th(\Gamma)$$

Proof. Suppose otherwise, $\exists \phi \in Th(Th(\Gamma)), \phi \notin Th(\Gamma)$.

$$\phi \in Th(Th(\Gamma)) \implies Th(\Gamma) \models \phi$$

$$\phi \notin Th(\Gamma) \implies \Gamma \not\models \phi \implies \exists \mathfrak{A}, \mathfrak{A} \models \Gamma, \mathfrak{A} \not\models \phi$$

$$\mathfrak{A} \models Th(\Gamma), \text{ contradiction}$$

□

Theorem 8.51.

$$Th(\Gamma) \text{ is inconsistent} \implies Th(\Gamma) \models \text{all sentences}$$

Definition 8.52 (Consistency of a theory). *Theory T is consistent if*

$$\forall \phi, \neg(\{\phi\} \subseteq T \wedge \{\neg\phi\} \subseteq T)$$

Definition 8.53 (Completeness of a theory). *Theory T is complete if*

$$\forall \phi, \{\phi, \neg\phi\} \cap T \neq \emptyset$$

Lemma 8.54.

$$Th(\mathfrak{A}) \text{ is consistent}$$

Proof. Suppose otherwise, $\phi, \neg\phi \in Th(\mathfrak{A}). \mathfrak{A} \models \phi \oplus \mathfrak{A} \models \neg\phi$, contradiction.

□

Lemma 8.55.

$$Th(\mathfrak{A}) \text{ is complete}$$

Proof. Let ϕ be a sentence,

- if $\mathfrak{A} \models \phi, \phi \in Th(\mathfrak{A})$
- if $\mathfrak{A} \not\models \phi, \mathfrak{A} \models \neg\phi, \neg\phi \in Th(\mathfrak{A})$

□

Lemma 8.56.

$$Th(\mathfrak{C}) \text{ is consistent}$$

Proof. Suppose otherwise, $\exists \phi, \phi, \neg\phi \in Th(\mathfrak{C})$

$$\forall \mathfrak{A} \in \mathfrak{C}, \mathfrak{A} \models \phi \oplus \mathfrak{A} \models \neg\phi$$

contradiction

□

Lemma 8.57. *$Th(\mathfrak{C})$ is not complete.*

Proof. A theory T is complete if for every sentence ψ , either $T \models \psi$ or $T \models \neg\psi$. To show $Th(\mathfrak{C})$ is incomplete, we can find a sentence ϕ such that $Th(\mathfrak{C}) \not\models \phi$ and $Th(\mathfrak{C}) \not\models \neg\phi$.

Let $\phi \equiv \exists x \exists y, R(x, y)$. Consider the following two structures in $\mathfrak{C}$:

1. Structure $\mathfrak{A} = (U_{\mathfrak{A}}, I_{\mathfrak{A}})$:

- $U_{\mathfrak{A}} = \{0, 1\}$
- $I_{\mathfrak{A}}(R) = \{(0, 1)\}$

In this structure, $R(0, 1)$ holds, therefore $\mathfrak{A} \models \phi$. This implies $Th(\mathfrak{C}) \not\models \neg\phi$.

2. Structure $\mathfrak{B} = (U_{\mathfrak{B}}, I_{\mathfrak{B}})$:

- $U_{\mathfrak{B}} = \{0, 1\}$
- $I_{\mathfrak{B}}(R) = \emptyset$

In this structure, no pair (x, y) satisfies R , therefore $\mathfrak{B} \models \neg\phi$. This implies $Th(\mathfrak{C}) \not\models \phi$.

Since there exists a structure in $\mathfrak{C}$ that satisfies ϕ and another that satisfies $\neg\phi$, the theory $Th(\mathfrak{C})$ contains neither. Thus, $Th(\mathfrak{C})$ is not complete. $\square$

Theorem 8.58. *Let T be a theory, $T \in RE$. If T is consistent and complete, $T \in REC$*

Proof. Let E be an enumerator for T . To decide if $\psi \in T$:

1. Start E to produce sentences $\sigma_1, \sigma_2, \sigma_3, \dots$
2. For each σ_i , check if $\sigma_i = \psi$ or $\sigma_i = \neg\psi$.
3. By completeness, one of these must eventually appear.
4. If ψ appears, accept. If $\neg\psi$ appears, reject.

Since the algorithm always halts, $T \in REC$. $\square$

8.8 Nelson-Oppen Theory Combination

The Nelson-Oppen method addresses the problem: given two theories T_1 and T_2 , how can we decide the satisfiability of a formula ϕ in the combined theory $T_1 \cup T_2$?

Definition 8.59 (Convex Theory). *A theory T is **convex** if for any conjunction of literals Γ , if $\Gamma \models_T \bigvee_{i=1}^n (x_i = y_i)$, then there exists some $j \in \{1, \dots, n\}$ such that $\Gamma \models_T (x_j = y_j)$.*

Prerequisites for Nelson-Oppen:

1. The signatures of the theories must be disjoint ($\Sigma_1 \cap \Sigma_2 = \emptyset$), except for the equality symbol ($=$).
2. The theories must be **stably infinite** (every satisfiable formula has a model with an infinite universe).
3. The formulas must be quantifier-free conjunctions of literals.

8.8.1 The Nelson-Oppen Algorithm

The procedure involves two main phases: Purification and Equality Propagation. **Step 1: Purification** The input formula ϕ is transformed into an equivalent formula $\phi_1 \wedge \phi_2$ such that each ϕ_i contains only symbols from Σ_i .

- This is achieved by introducing auxiliary variables to replace "alien" terms.
- For example, in $f(g(x)) = x$ where $f \in \Sigma_1$ and $g \in \Sigma_2$, we replace $g(x)$ with a new variable v , resulting in $f(v) = x \wedge v = g(x)$.

Step 2: Equality Propagation The individual theory solvers communicate by sharing information about equalities between shared variables (variables that appear in both ϕ_1 and ϕ_2).

- Solver D_1 checks if ϕ_1 is satisfiable in T_1 .
- Solver D_2 checks if ϕ_2 is satisfiable in T_2 .
- If either is unsatisfiable, the combined formula is unsatisfiable.
- If D_1 implies an equality between shared variables $x = y$, this equality is added to ϕ_2 .
- If D_2 implies an equality, it is added to ϕ_1 .
- This continues until a contradiction is found or no more equalities can be inferred.

8.9 Complexity and Completeness

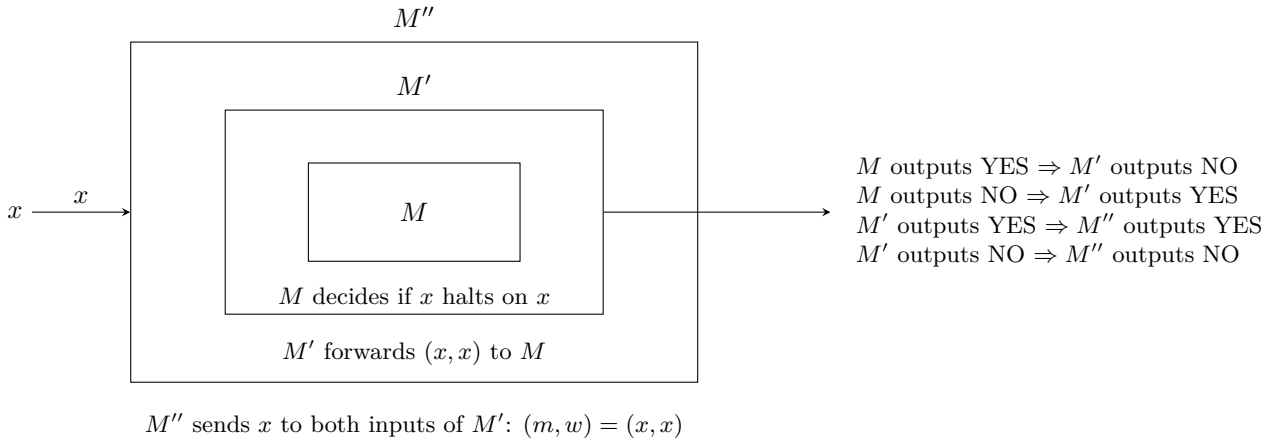
- **Completeness:** For convex theories (like the theory of equality or linear arithmetic over reals), propagating single equalities is sufficient. For non-convex theories (like linear arithmetic over integers), the solvers may need to perform case splits on disjunctions of equalities, leading to higher complexity.
- **Decidability:** If the individual satisfiability problems for T_1 and T_2 are decidable, then the Nelson-Oppen procedure provides a decision procedure for the combined theory $T_1 \cup T_2$.

9 Computability

9.1 Halting problem

Theorem 9.1. *The halting problem is undecidable*

Proof.



Now consider $M''(M'')$.

Case 1: $M''(M'') = NO$

□

Definition 9.2 (Recursively Enumerable). $L \subseteq \Sigma^*$ is recursively enumerable (RE), if there is a Turing machine $\mathcal{M}$, such that $L = L(\mathcal{M})$. Other equivalent terms for an RE language are:

- Semi-decidable
- Turing recognizable

Definition 9.3 (Recursive). $L \subseteq \Sigma^*$ is a recursive (REC) language if there is a Turing machine $\mathcal{M}$, such that $L = L(\mathcal{M})$, and $\mathcal{M}$ is terminating. Recursive languages are also called decidable

Theorem 9.4.

$$L_H = \{\langle \mathcal{M}, w \rangle \mid w \text{ terminates on } \mathcal{M}\} \in RE$$

Proof. We can construct a TM $\mathcal{M}_{Halt}$ which is the turing machine that returns YES if its input machine terminates on input string.

```

 $\mathcal{M}_{Halt}(\mathcal{M}, w)$ :
   $\mathcal{M}(w)$ 
  return YES
  
```

This suffice to show that $L_H \in RE$

$L_H \notin REC$

Similar to how we prove undecidability of the Halting problem □

Theorem 9.5 (Membership).

$$MP = \{\langle \mathcal{M}, w \rangle \mid w \text{ is accepted by } \mathcal{M}\} \in RE$$

Proof. We can construct TM

```

 $\mathcal{M}_{UTM}(\mathcal{M}, w):$ 
   $b = \mathcal{M}(w)$ 
  return  $b$ 

```

□

Quick Reference: Reduction Recipes

If you are stuck on the construction logic, these are the standard "wrappers" for M' :

1. **To show X is "Large" (e.g., Non-Empty):** Construct $M'(x)$ to simulate $M(w)$. If it halts, **Accept**. *Result:* $L(M')$ is either Σ^* (if M halts) or $\emptyset$ (if it loops).
2. **To show X is "Small" (e.g., Empty or Finite):** To reduce $\overline{L_{Halt}}$ to L_{Empty} , use the same construction as above. *Logic:* M loops on $w \iff L(M') = \emptyset \iff \langle M' \rangle \in L_{Empty}$.

Goal: Show X is...	Hierarchy	Source (A)	Requirement
Undecidable	$\notin \Delta_1$	L_{Halt}	$A \leq_m X$
Not in RE	$\notin \Sigma_1$	$\overline{L_{Halt}}$	$A \leq_m X$
Not in co-RE	$\notin \Pi_1$	L_{Halt}	$A \leq_m X$
Not in RE $\cup$ co-RE	$\notin \Sigma_1 \cup \Pi_1$	L_{Total}	$A \leq_m X$

Table 1: Strategic mapping for complexity proofs.

3. **To involve all strings (e.g., Totality):** Construct $M'(x)$ to simulate $M(w)$ for exactly $|x|$ steps. This is often used for more complex Π_2 proofs.

The "Direction" Rule

Always remember: Hard $\leq$ New.

- You are "borrowing" the difficulty of the known language to prove the unknown one is just as bad.
- If the reduction f maps "Yes" instances to "Yes" instances and "No" to "No", the complexity flows from the source to the target.

Theorem 9.6.

$$L \in REC \iff \bar{L} \in REC$$

Proof. $L \in REC \implies \exists \mathcal{M}, L = L(\mathcal{M}), \mathcal{M}$ is terminating, we can construct another TM

```

 $\mathcal{M}'(w):$ 
   $b = \mathcal{M}(w)$ 
  return  $\neg b$ 

```

It is easy to show that $L(\mathcal{M}') = \bar{L}$, and $\mathcal{M}'$ is terminating. □

Theorem 9.7.

$$L \in RE \wedge \bar{L} \in RE \implies L \in REC$$

Proof. Consider then TM for L and $\bar{L}$, $\mathcal{M}_1$ and $\mathcal{M}_2$, we can construct a new TM

```

 $\mathcal{M}(w)$ :
  for  $i > 0$ :
    run the  $i^{th}$  step of  $\mathcal{M}_1$ 
    if  $\mathcal{M}_1$  reaches  $q_{\mathcal{M}_1, Acc}$ 
      return YES
    run the  $i^{th}$  step of  $\mathcal{M}_2$ 
    if  $\mathcal{M}_2$  reaches  $q_{\mathcal{M}_2, Acc}$ 
      return NO

```

Since $\forall w, \forall L \subseteq \Sigma^*, w \in L \vee w \in \bar{L}$, w must be accepted by either $\mathcal{M}_1$ or $\mathcal{M}_2$, therefore $\mathcal{M}$ is terminating. $\square$

9.2 Reduction

Definition 9.8. $L_1 \leq_m L_2$ if there is a computable function f , such that

$$\begin{aligned}
 w \in L_1 &\implies f(w) \in L_2 \\
 w \notin L_1 &\implies f(w) \notin L_2
 \end{aligned}$$

Theorem 9.9.

$$L_1 \leq_m L_2, L_2 \in REC \implies L_1 \in REC$$

Theorem 9.10.

$$L_1 \leq_m L_2, L_2 \in RE \implies L_1 \in RE$$

Proof. $\square$

Theorem 9.11.

$$L_1 \leq_m L_2, L_1 \in REC \not\implies L_2 \in REC$$

Proof. Consider the following recursive language:

$$L_1 = \{\langle \mathcal{M}, w \rangle \mid w \text{ terminates on } \mathcal{M} \text{ in 10 steps}\}$$

and

$$L_H = \{\langle \mathcal{M}, w \rangle \mid w \text{ terminates on } \mathcal{M}\}$$

We have shown that $L_H \in RE \setminus REC$. We can construct the following terminating TM to define a computable function f

```

 $\mathcal{M}_f(\langle \mathcal{M}, w \rangle)$ :
  run  $\mathcal{M}_1$  on  $w$  for 10 steps
  if output is YES:
    return  $\langle \mathcal{M}, w \rangle$ 
  else:
    return  $\langle NH, w \rangle$ 

```

Where NH is the encoding of a TM that loops forever on all input.

It is easy to show that

$$\begin{aligned}
 \langle \mathcal{M}, w \rangle \in L_1 &\implies f(\langle \mathcal{M}, w \rangle) \in L_H \\
 \langle \mathcal{M}, w \rangle \notin L_1 &\implies f(\langle \mathcal{M}, w \rangle) \notin L_H
 \end{aligned}$$

But $L_H \notin REC$ as we have previously shown. Contradiction $\square$

9.3 RE-hardness

There are languages beyond the RE class. For example

Theorem 9.12.

$$\bar{L}_H \notin RE$$

Proof. Suppose otherwise, $\bar{L}_H \in RE, L_H \in RE \implies L_H \in REC$, contradiction $\square$

Definition 9.13 (RE-hard). *L is RE-hard, if*

$$\forall L' \in RE, L' \leq_m L$$

Example 9.14.

$$MP = \{\langle \mathcal{M}, w \rangle \mid w \in L(\mathcal{M})\}$$

$$L_{Halt} \leq_m MP$$

Proof. We construct a computable function $f(\langle M, w \rangle) = \langle M', w \rangle$. The machine M' is defined as follows:

$M'(x)$:

Simulate M on input w
Accept // Only reached if $M(w)$ halts

Clearly, M halts on $w \iff M'$ accepts x (for any x). Thus $\langle M, w \rangle \in L_{Halt} \iff \langle M', w \rangle \in MP$. $\square$

Similarly reduction can show some languages are not in RE, if they are at least as hard as a known not in RE language

Example 9.15. $\bar{MP} \notin RE$

Proof by Reduction: $\bar{K} \leq_m \bar{MP}$

Given $\bar{K} = \{\langle M \rangle \mid \langle M \rangle \notin L(M)\}$ is not in RE. Define a computable transformation $f(\langle M \rangle) = \langle M', \epsilon \rangle$, where M' is:

$M'(x)$:

Simulate M on input $\langle M \rangle$
Accept // Only reached if M accepts $\langle M \rangle$

1. If $\langle M \rangle \in \bar{K}$, then M does not accept $\langle M \rangle$. M' will loop/reject on the simulation, so $L(M') = \emptyset$. Thus, $\epsilon \notin L(M') \implies \langle M', \epsilon \rangle \in \bar{MP}$.

2. If $\langle M \rangle \notin \bar{K}$, then M accepts $\langle M \rangle$. M' will reach the "Accept" command for any input. Thus, $\epsilon \in L(M') \implies \langle M', \epsilon \rangle \notin \bar{MP}$.

Conclusion: $\langle M \rangle \in \bar{K} \iff f(\langle M \rangle) \in \bar{MP}$.

9.4 Validity of FOL

Definition 9.16.

$$VALID : \{\varphi \mid \varphi \text{ is valid} \}$$

Theorem 9.17. $VALID \in RE$

Theorem 9.18 (Church-Turing Theorem). *VALID is RE-hard*

10 Second Order Logics

10.1 Syntax of Second Order Logics

in second order logics, terms are defined the same as in FOL.

$$\Sigma = (C, F, R)$$

$$V = FOV \cup SOV$$

where FOV is the set containing variables in FOL, and SOV contains the relation variables, for example

$$SOV = \{X^2, Y^1, \dots\}$$

where the superscripts denote the arity of these relation variable.

Definition 10.1.

$$\varphi := t = t \mid R^k(t_1, \dots, t_k) \mid X^k(t_1, \dots, t_k) \mid \neg\varphi \mid \varphi \vee \varphi \mid \exists x, \varphi \mid \forall x, \varphi \mid \exists X, \varphi \mid \forall X, \varphi$$

Example 10.2.

$$\phi \equiv \forall x, \exists X^1, X(x)$$

This formula means for all x in the universe, there exist such a unary relation, such that x is in the relation. This formula is valid as we can always interpret $X = U$.

Semantics of SOL

Similar to FOL, the semantics of SOL is defined by a structure and an assignment (with free variables)

$$\mathfrak{A} = (U, I)$$

Definition 10.3. for FOL variable:

$$\alpha(x) \in U$$

for SOL variables:

$$\alpha(X^k) \subseteq U^k$$

Similar to FOL, model relation is defined recursively:

$$\begin{aligned} \mathfrak{A}, \alpha \models X^k(t_1, \dots, t_k) &\iff Val_{\mathfrak{A}, \alpha}(t_1), \dots, Val_{\mathfrak{A}, \alpha}(t_k) \in \alpha(X) \\ \mathfrak{A}, \alpha \models \exists X^k, \phi &\iff \text{there is a set of } k\text{-ary tuples of } U, S \subseteq U^k \\ &\text{such that } \mathfrak{A}, \alpha[X \mapsto S] \models \phi \\ \mathfrak{A}, \alpha \models \forall X^k, \phi &\iff \text{for all sets of } k\text{-ary tuples of } U, S \subseteq U^k \\ &\text{such that } \mathfrak{A}, \alpha[X \mapsto S] \models \phi \end{aligned}$$

10.2 Expressiveness of SOL

Definition 10.4. Let ϕ be a sentence(no free variables), we can define the set of structures that models ϕ as

$$\llbracket \phi \rrbracket = \{\mathfrak{A} \mid \mathfrak{A} \models \phi\}$$

We have proven by compactness of FOL, it is impossible to express the set of all finite/infinite structures with FOL formula. However we can do so in SOL.

10.2.1 Finite structures

Theorem 10.5. There exist a SOL formula ϕ_{fin} such that

$$\llbracket \phi_{fin} \rrbracket = \{\mathfrak{A} \mid |U_{\mathfrak{A}}| < \infty\}$$

We need to first find some unique property of finite sets

Theorem 10.6. *Let S be a set, S is finite $\iff \forall f : S \mapsto S$ that are injective, f is also surjective*

Definition 10.7 (structure classes defined by non-sentence). *For formula ϕ , where $FVar(\phi) = \{\bar{x}\}$*

$$\llbracket \phi(\bar{x}) \rrbracket = \{(\mathfrak{A}, \bar{e} \in U^k) \mid \mathfrak{A}, \alpha[\bar{x} \mapsto \bar{e}] \models \phi\}$$

We can first use SOL to define function-like relations.

Definition 10.8. *A binary relation $S^2 \subseteq U^2$ is function-like, if*

1. $\forall u \in U, \exists v, (u, v) \in S$
2. $\forall u, v_1, v_2, (u, v_1) \in S \wedge (u, v_2) \in S \rightarrow (v_1 = v_2)$

This is a literal translation of all elements in domain has a value to be mapped to, and no element maps to 2 different elements

These constraints are be expressed with SOL with free variable:

$$\phi_{func}(X^2) \equiv \forall x \exists y X(x, y) \wedge \forall x \forall y \forall z (X(x, y) \wedge X(x, z) \rightarrow (y = z))$$

$$\phi_{injection}(X^2) \equiv \phi_{func}(X^2) \wedge \forall x \forall y \forall z (X(x, y) \wedge X(y, z) \rightarrow (x = y))$$

$$\phi_{surjection}(X^2) \equiv \phi_{func}(X^2) \wedge \forall y \exists x X(x, y)$$

Therefore, we can use a SOL formula to express finite structures:

$$\phi_{fin} \equiv \forall X^2 (\phi_{injection}(X^2) \rightarrow \phi_{surjection}(X^2))$$

Follow the same logic as how we proved that it is impossible to express such concept in FOL due to compactness, we can prove that in SOL, compactness does not hold.

10.2.2 Reachability

Consider the following signature:

$$\Sigma = (\{s, t\}, \emptyset, \{E^2\})$$

$$\mathfrak{C}_{s \rightsquigarrow t} = \{\mathfrak{A} \mid I(s) \text{ has a path using } I(E) \text{ to } T(t)\}$$

We have shown this is not expressable in FOL. To express this concept, we need to characterize reachability as a property, which is a binary relation that is

1. reflexive
2. transitive

$$\phi_{reflexive}(X^2) \equiv \forall x X(x, x)$$

$$\phi_{transitive}(X^2) \equiv \forall x \forall y \forall z X(x, y) \wedge X(y, z) \rightarrow X(x, z)$$

$$\llbracket \phi_{s \rightsquigarrow t} \rrbracket \equiv \left\{ (\mathfrak{A} = (U, I), R \subseteq U^2) \mid \begin{array}{l} \exists w_0, \dots, w_k \in U, \text{ such that:} \\ (s, w_0) \in I(E), \\ \forall i \in \{0, \dots, k-1\}, (w_i, w_{i+1}) \in I(E), \\ (w_k, t) \in I(E) \end{array} \right\}$$

However we can artificially inflate the set to make it still satisfy the semantics of the formula, but contain more pairs that are not connected. We can use the same technique used to define the set of WFF, which is the **smallest** set.

Definition 10.9. *The set of reachable pairs $\mathfrak{R}$ is the **smallest** set of pairs that are*

1. $E \subseteq S$

2. transitive and reflexive

First we define the concept of closure

$$\phi_{closed}(X^2) \equiv \phi_{reflexive}(X^2) \wedge \phi_{transitive}(X^2) \wedge E \subseteq X$$

the concept of E subsetting X can also be written in SOL as a shorthand

$$\subseteq (X^2) \equiv \forall x \forall y E(x, y) \rightarrow X(x, y)$$

Then we can formally define the relation reachable:

$$\phi_{reach}(X^2) \equiv \phi_{closed}(X^2) \wedge \forall Y^2 (\phi_{closed}(Y^2) \rightarrow X^2 \supseteq Y^2)$$

10.2.3 Cyclic graph

A cyclic graph is a graph that contains at least one cycle.

Irreflexive reachability can be defined similarly as reachability, but without the reflexive clause.

$$\phi_{irreflexive_closed} = \phi_{transitive}(X^2) \wedge E \subseteq X$$

$$\phi_{irreflexive_reach}(X^2) \equiv \phi_{irreflexive_closed} \wedge \forall Y^2 (\phi_{irreflexive_closed}(Y^2) \rightarrow X^2 \supseteq Y^2)$$

$$\phi_{cyclic} \equiv \exists x \exists X^2 (\phi_{irreflexive_reach}(X^2) \wedge X(x, x))$$

Alternatively we can use reachability and additional constraints to enforce reflexivity:

$$\exists x \exists y (\exists X^2 \phi_{reach}(X^2) \wedge E(x, y) \wedge E(y, x))$$

10.2.4 Even

All structures that have finite and even number of elements in the universe

$$\phi_{disjoint}(X_1, X_2) \equiv \forall x X_1(x) \leftrightarrow \neg(X_2(x))$$

$$\phi_{exhaustive}(X_1, X_2) \equiv \forall x X_1(x) \vee X_2(x)$$

Before we defined a formula to describe relations that are injection-function-like, now we define a formula that describe, there exists a binary relation, between the sets defined by 2 relations.

$$\begin{aligned} \phi_{injection}(X, Y) \equiv \exists F \Big[& (\forall x : X(x) \rightarrow \exists y : (Y(y) \wedge F(x, y))) \wedge \\ & (\forall x, y_1, y_2 : X(x) \wedge Y(y_1) \wedge Y(y_2) \wedge F(x, y_1) \wedge F(x, y_2) \rightarrow y_1 = y_2) \wedge \\ & (\forall x_1, x_2, y : X(x_1) \wedge X(x_2) \wedge Y(y) \wedge F(x_1, y) \wedge F(x_2, y) \rightarrow x_1 = x_2) \Big] \end{aligned}$$

$$\begin{aligned} \phi_{surjection}(X, Y) \equiv \exists F \Big[& (\forall x : X(x) \rightarrow \exists y : (Y(y) \wedge F(x, y))) \wedge \\ & (\forall x, y_1, y_2 : X(x) \wedge Y(y_1) \wedge Y(y_2) \wedge F(x, y_1) \wedge F(x, y_2) \rightarrow y_1 = y_2) \wedge \\ & (\forall y : Y(y) \rightarrow \exists x : (X(x) \wedge F(x, y))) \Big] \end{aligned}$$

The first and second lines define totality and function-like property.

$$\phi_{bijection}(X_1, X_2) \equiv \phi_{injection}(X_1, X_2) \wedge \phi_{surjection}(X_1, X_2)$$

$$\phi_{even} \equiv \exists X_1^1 \exists X_2^1 (\phi_{disjoint}(X_1, X_2) \wedge \phi_{bijection}(X_1, X_2) \wedge \phi_{exhaustive}(X_1, X_2))$$

10.2.5 Bipartite graph

$$\begin{aligned} \phi_{bipartite} \equiv \exists X_1^1, X_2^1 \Big[& \phi_{disjoint}(X_1, X_2) \wedge \phi_{exhaustive}(X_1, X_2) \wedge \\ & \forall x, y (E(x, y) \rightarrow \neg(X_1(x) \wedge X_1(y)) \wedge \neg(X_2(x) \wedge X_2(y))) \Big] \end{aligned}$$

We can observe that n-partite graphs are exactly thos graphs that are n-colorable, they are identical structures

10.2.6 Connectedness

A graph is connected if every pair of vertices has a path between them. Since you have already defined $\phi_{reach}(X^2)$, we can use it to define a connected graph.

$$\phi_{connected} \equiv \exists X^2 (\phi_{reach}(X^2) \wedge \forall u \forall v X(u, v))$$

Alternatively, we can express it by stating that there is no non-trivial partition of the graph that "cuts" all edges:

$$\phi_{connected} \equiv \forall X^1 ((\exists x X(x) \wedge \exists y \neg X(y)) \rightarrow \exists u \exists v (X(u) \wedge \neg X(v) \wedge E(u, v)))$$

10.2.7 Well-Ordering

In FOL, we can define a total order, but we cannot express that a relation is a well-order (every non-empty set has a least element) because we cannot quantify over sets. In SOL, this is straightforward: Let $\phi_{total}(R^2)$ be the FOL formula for a total order.

$$\phi_{well_order}(R^2) \equiv \phi_{total}(R^2) \wedge \forall X^1 (\exists x X(x) \rightarrow \exists m (X(m) \wedge \forall y (X(y) \rightarrow R(m, y))))$$

10.2.8 Hamiltonian Graphs

A graph is Hamiltonian if it contains a cycle that visits every vertex exactly once. While this property is non-expressible in FOL, it is easily captured in SOL. By Fagin's Theorem, this existential second-order formula (Σ_1^1) also serves as a proof that the problem is in NP. We define a binary relation H^2 to act as a successor relation. The formula $\phi_{hamiltonian}$ is defined as:

$$\begin{aligned} \phi_{hamiltonian} \equiv \exists H^2 \Big[& \underbrace{\phi_{inj}(H^2) \wedge \phi_{surj}(H^2)}_{\text{Permutation}} \wedge \underbrace{\forall x \forall y (H(x, y) \rightarrow E(x, y))}_{\text{Sub-graph of } E} \\ & \wedge \underbrace{\forall X^1 ((\exists x X(x) \wedge \forall u \forall v (X(u) \wedge H(u, v) \rightarrow X(v))) \rightarrow \forall x X(x))}_{\text{Connectivity (Induction Axiom)}} \Big] \end{aligned}$$

Logical Breakdown: Permutation: $\phi_{inj} \wedge \phi_{surj}$ ensures that every vertex has exactly one incoming edge and one outgoing edge in H . This forces H to be a collection of one or more disjoint cycles. Sub-graph: Ensures that H only uses edges that actually exist in the original graph E . Induction Axiom: This is the "secret sauce." It states that any set X closed under the successor relation H must be the entire universe U . This prevents the graph from being split into multiple smaller disjoint cycles (sub-cycles), forcing a single cycle that covers all vertices.

10.2.9 Well-Foundedness

A relation E is well-founded if there are no infinite descending chains $x_1 \leftarrow x_2 \leftarrow x_3 \dots$. This is equivalent to saying every non-empty set has an E -minimal element.

$$\phi_{well_founded} \equiv \forall X^1 (\exists x X(x) \rightarrow \exists m (X(m) \wedge \forall y (X(y) \rightarrow \neg E(y, m))))$$

10.2.10 Graph Isomorphism

Another powerful use of SOL is expressing whether two graphs are isomorphic. Given two graphs $\mathfrak{A} = (U_A, E_A)$ and $\mathfrak{B} = (U_B, E_B)$, they are isomorphic if there exists a bijection $f : U_A \rightarrow U_B$ that preserves the edge relation.

$$\begin{aligned} \phi_{iso} \equiv \exists F^2 \Big[& \phi_{bijection}(F^2) \wedge \\ & \forall u \forall v (E_A(u, v) \leftrightarrow \exists u' \exists v' (F(u, u') \wedge F(v, v') \wedge E_B(u', v'))) \Big] \end{aligned}$$